

คู่มือโปรแกรมเมอร์
ระบบค้นหาเส้นทางอัจฉริยะในกรุงเทพ (ส่วนแสดงผล)
Intelligent Bangkok Traffic Router (Front-end)

นายวุฒินัย กาญจนสร
นายสันหิ ภัทรวิทย์

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต
ภาควิชาวิศวกรรมคอมพิวเตอร์
คณะวิศวกรรมคอมพิวเตอร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา 2546

ตารางฐานข้อมูล

ฐานข้อมูลที่ใช้ในระบบของส่วนการแสดงผลนี้ใช้ MySQL เป็นฐานข้อมูล ซึ่งประกอบไปด้วยตารางทั้งหมด 7 ตารางคือ

ตาราง Member

เป็นตารางที่เก็บรายละเอียดของสมาชิกของระบบ โดยมีฟิลด์ต่างๆ ดังนี้คือ

ฟิลด์	รายละเอียด
id	หมายเลขลำดับสมาชิก
usr	เป็นชื่อที่ใช้ในการล็อกอิน (login) เข้าสู่ระบบสมาชิก
pass	รหัสที่ใช้การเข้ารับบริการของระบบ
name	ชื่อและนามสกุลของสมาชิก
email	อีเมลที่ใช้ในการติดต่อ
district	เขตพื้นที่ของกรุงเทพมหานครฯ เช่น ลาตกระบ้ง, พระโขนง
address	ที่อยู่ของสมาชิก
mobile	หมายเลขโทรศัพท์เคลื่อนที่ของสมาชิก
Provider	ผู้ให้บริการโทรศัพท์เคลื่อนที่
Mo_usr	ชื่อที่ใช้ในการล็อกอินของโทรศัพท์เพื่อส่ง SMS
Mo_pass	รหัสผ่านที่ใช้เพื่อส่ง SMS
Start1	จุดเริ่มต้นที่ 1 ที่สมาชิกต้องการให้ระบบค้นหาเส้นทางให้
dest1	จุดสิ้นสุดที่ 1 ที่สมาชิกต้องการให้ระบบค้นหาเส้นทางให้
Start2	จุดเริ่มต้นที่ 2 ที่สมาชิกต้องการให้ระบบค้นหาเส้นทางให้
Dest2	จุดเริ่มต้นที่ 2 ที่สมาชิกต้องการให้ระบบค้นหาเส้นทางให้

แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง Member

ตาราง Matching

เป็นตารางที่ใช้ทำการเปรียบเทียบเพื่อแปลงจากชื่อบริเวณต่างๆ ซึ่งเป็นได้ทั้งชื่อถนน, สถานที่และชื่อแยก ให้กลายเป็นชื่อแยกทั้งหมดซึ่งก็คือจุดที่สามารถเกิดการเลี้ยวเปลี่ยนเส้นทางการเดินทางได้ เช่น บริเวณยูเทิร์น (U-turn) ของถนน และสี่แยก

เนื่องจากว่าในการทำงานของส่วนที่เป็นระบบค้นหาเส้นทางอัจฉริยะส่วนประมวลผลนั้น จะเป็นการคำนวณจากโหนด (node) ดังกล่าวทั้งสิ้น ดังนั้นเพื่อความยืดหยุ่นในการใช้งานของผู้บริการให้มีความหลากหลาย จึงต้องทำการแปลงก่อน แล้วจึงส่งต่อไปให้ส่วนประมวลผลทำงานต่อไป

ฟิลด์	รายละเอียด
word	ข้อบริเวณซึ่งเป็นได้ทั้งข้อถนน, สถานที่และข้อแยก
inter1	เป็นชื่อที่ใช้ในการล็อกอิน (login) เข้าสู่ระบบสมาชิก
inter2	รหัสที่ใช้การเข้ารับบริการของระบบ

แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง *Matching*

ตาราง Report

เป็นตารางที่ใช้ในการรายงานอุบัติเหตุหรือเกิดสิ่งกีดขวางการจราจรที่เกิดขึ้น โดยการค้นหาจะเริ่มจากชื่อถนนหรือชื่อเขตของกรุงเทพมหานคร

ฟิลด์	รายละเอียด
street	ชื่อถนนที่เกิดอุบัติเหตุหรือเกิดสิ่งกีดขวางการจราจร
district	ชื่อเขตที่เกิดอุบัติเหตุหรือเกิดสิ่งกีดขวางการจราจร
comment	รายงานอุบัติเหตุ หรือเกิดสิ่งกีดขวางการจราจร
date	วันที่ที่เกิดรายงานนั้นๆ

แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง *Report*

ตาราง District

เก็บรายชื่อของเขตต่างๆภายในกรุงเทพมหานคร เพื่อการเปลี่ยนแปลงในภายหลัง เช่น การเพิ่มหรือลบเขตจะสามารถทำได้ง่ายขึ้นโดยกระทำที่จุดเดียว

ฟิลด์	รายละเอียด
id	หมายเลขลำดับของเขตภายในกรุงเทพมหานคร โดยเรียงตามตัวอักษร
distr_name	รายชื่อเขตของกรุงเทพมหานคร

แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง *Report*

ตาราง Board_quest

ใช้เก็บรายละเอียดของเว็บบอร์ด ซึ่งตารางนี้จะเก็บเกี่ยวกับกระทู้หลัก (หัวข้อกระทู้) ทั้งหมดที่มีของเว็บบอร์ด แต่จะไม่ได้ทำการเก็บเกี่ยวกับรายละเอียดของการตอบกระทู้

ฟิลด์	รายละเอียด
id	หมายเลขลำดับของหัวข้อกระทู้ โดยเรียงตามเวลาที่สร้างก่อนหลัง
topic	ชื่อหัวข้อกระทู้
name	ชื่อผู้ตั้งกระทู้ (ชื่อที่ใช้ในการล็อกอินของสมาชิก)
email	อีเมลล์ของผู้ตั้งกระทู้ที่ได้ให้ไว้ในฐานข้อมูลสมาชิก

detail	รายละเอียดของหัวข้อกระทู้
ip	หมายเลขไอพีแอดเดรส (IP address) ของเครื่องที่ทำการสร้างกระทู้
datepost	วันที่และเวลาที่สร้างกระทู้
lastpost	วันที่และเวลาครั้งล่าสุดที่ได้มีการเปลี่ยนแปลงกระทู้ (ตอบหรือสร้าง)
Lastname	ชื่อล็อกอินของผู้ที่ทำการเปลี่ยนแปลงครั้งล่าสุด
View	จำนวนของผู้ที่เข้ามาดูกระทู้นี้
ans	จำนวนผู้ที่ทำการตอบกระทู้

แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง Board_quest

ตาราง Board_ans

ใช้เก็บรายละเอียดของเว็บบอร์ด ซึ่งตารางนี้จะเก็บเกี่ยวกับการโพสต์ตอบกระทู้ โดยเชื่อมโยงกับตาราง Board_quest โดยใช้หมายเลข id_quest ของตารางนี้ให้ตรงกับหมายเลข id ในตาราง Board_quest หากตรงกัน หมายถึงคำตอบกระทู้นี้เป็นของกระทู้นั้นๆ

ฟิลด์	รายละเอียด
id_ans	หมายเลขของคำตอบกระทู้
id_quest	หมายเลขของกระทู้ซึ่งตรงกันกับของตาราง Board_quest
name	ชื่อล็อกอินของผู้ตอบกระทู้
email	อีเมลล์ของผู้ตอบกระทู้ที่ได้ให้ไว้ในฐานข้อมูล
ip	หมายเลขไอพีแอดเดรส (IP address) ของเครื่องที่ทำการตอบกระทู้
datepost	วันที่และเวลาที่ได้ทำการตอบกระทู้

แสดงชื่อฟิลด์และสิ่งที่เก็บภายในตาราง Board_quest

ตาราง Mapinfo

เป็นตารางที่จะถูกเรียกใช้งานโดย Applet เพื่อใช้ในการแสดงแผนที่ และสร้างจุดที่แสดงให้เห็นถึงบริเวณที่มีโหนด (จุดที่สามารถเกิดการเลี้ยวเพื่อเปลี่ยนเส้นทางการเดินทางได้)

โดยที่ applet จะนำค่า x และ y จากฐานข้อมูลมาทำการวาดโหนด ถ้า applet ทำการแสดงผลภาพโดยผู้ก็จะทำการวาดโหนดที่มีฟิลด์ image ตรงกับชื่อรูปนั้นๆ

ฟิลด์	รายละเอียด
id	หมายเลขของโหนด
node	ชื่อโหนด
x_coord	ค่าตามแกน x ของโหนดเรียงจากซ้ายไปขวาโดยเริ่มที่ 0 มีค่าถึง 600
y_coord	ค่าตามแกน y ของโหนดเรียงจากบนลงล่างโดยเริ่มที่ 0 มีค่าถึง 600

junction	จำนวนแยกเส้นทางที่สามารถไปได้ของโหนด (ยูเทิร์นมีค่าเป็น 0)
image	ชื่อรูปภาพของแผนที่ที่จะทำการเรียกใน applet
default_district	เป็นชื่อเขตที่ใช้ในการบ่งบอกให้ทำการแสดงแผนที่ของเขตนั่นๆ

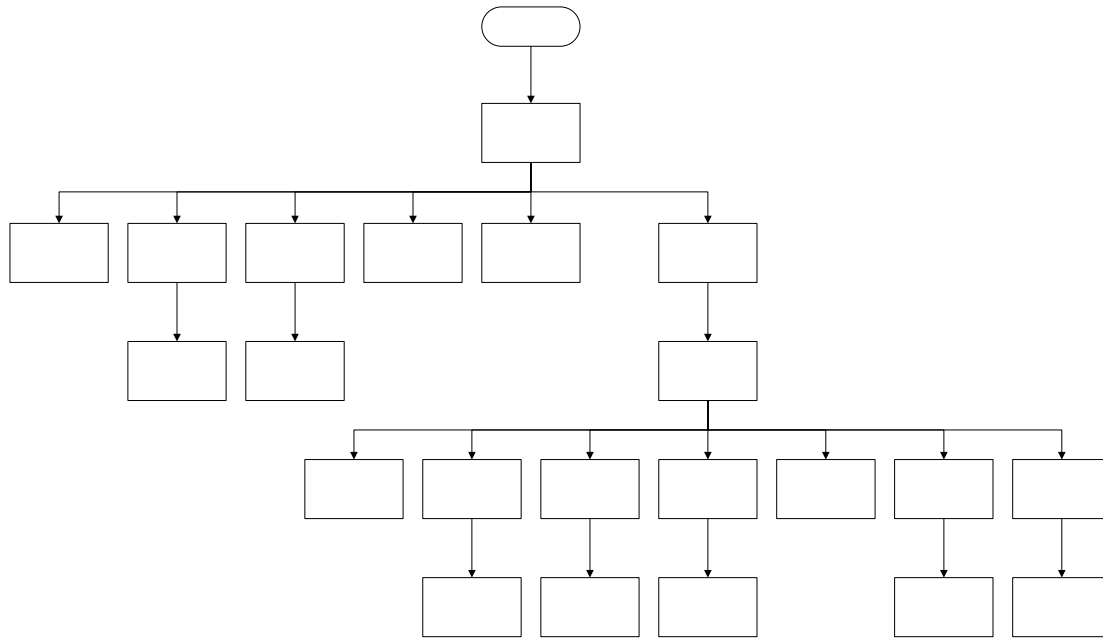
แสดงชื่อไฟล์และสิ่งที่เก็บภายในตาราง Board_quest

ขั้นตอนการทำงานส่วนแสดงผลของเว็บไซต์

การทำงานของส่วนเว็บไซต์จะอยู่บนเครื่องคอมพิวเตอร์ โดยใช้สคริปต์ JSP ในการเขียน โดยใช้ Tomcat เป็นเว็บเซิร์ฟเวอร์ และติดต่อกับฐานข้อมูลซึ่งก็คือ MySQL โดยผ่านทาง JDBC โดยไดรเวอร์ (Driver) ที่ชื่อว่า “org.gjt.mm.mysql.Driver” ซึ่งเครื่องที่เป็นเว็บเซิร์ฟเวอร์และเซิร์ฟเวอร์ฐานข้อมูลเป็นเครื่องเดียวกัน ลักษณะการทำงานจะแบ่งเป็นแต่ละเพจดังนี้ คือ

1. หน้าแรกหรือโฮมเพจ (home.jsp)
2. หน้าสมัครสมาชิกใหม่ (register.jsp)
3. หน้าแก้ไขข้อมูลส่วนตัวของสมาชิก (edit.jsp และ edit_cont.jsp)
4. หน้าล็อกอินหรือหน้าเข้าสู่ระบบสมาชิก (login.jsp)
5. หน้าล็อกเอาต์ออกจากระบบ (logout.jsp)
6. หน้าแสดงแผนที่และรายละเอียดของเส้นทางจราจร (map.jsp และ member_map.jsp)
7. หน้าค้นหาเส้นทางจราจร (routing.jsp และ member_routing.jsp)
8. หน้าแสดงรายงานอุบัติเหตุและสิ่งกีดขวางการจราจร (report.jsp และ member_report.jsp)
9. หน้าแสดงรายงานสถิติวิทยุออนไลน์ จส.100 (radio.jsp และ member_radio.jsp)
10. หน้าต้อนรับเข้าสู่ระบบสมาชิกหลังทำการล็อกอิน (member_section.jsp)
11. หน้าแสดงรายชื่อกระทู้ทั้งหมดของเว็บบอร์ด (webboard.jsp)
12. หน้าสร้างกระทู้ใหม่หรือหน้าโพสต์ (addnew.jsp)
13. หน้าแสดงและตอบกระทู้ (view.jsp)
14. หน้าแสดงความคิดเห็นของเซิร์ฟเวอร์ (error.jsp)
15. ไฟล์เก็บส่วนหัวและแถบเมนูด้านข้าง (header.jsp, member_header.jsp และ member_left.jsp)
16. โปรแกรม Java Applet ที่ใช้แสดงแผนที่ (Map.class)

หน้าที่ขึ้นต้นด้วย “member” ทั้งหมดและหน้าที่เกี่ยวกับเว็บบอร์ดจะมีตรวจสอบก่อนการเข้ามาดูทุกครั้งว่าบุคคลนั้นๆได้ผ่านการล็อกอินมาอย่างถูกต้องแล้วหรือยัง และจะมีการเรียกใช้งาน member_left.jsp และ member_header.jsp ซึ่งเป็นส่วนเมนูที่อยู่ทางด้านซ้ายและด้านบนของเว็บเพจตามลำดับ ส่วนหน้าเว็บเพจอื่นๆจะเรียกใช้ header.jsp ทุกๆเพจ ดังนั้นการที่จะแก้หน้าตาของเว็บเพจจึงสามารถกระทำได้ที่เดียว



แสดงลำดับการใช้งานของเว็บเพจ

และเมื่อเกิดข้อผิดพลาดขึ้น สำหรับทุกๆเพจจะเรียกไปยัง error.jsp

หน้าแรกหรือโฮมเพจ (home.jsp)

มีการแสดงลิงค์ไปยังเพจต่างๆภายในเว็บไซต์ และยังรวบรวมลิงค์ที่เกี่ยวข้องกับการจราจรภายในเขตกรุงเทพมหานครด้วย



Map.jsp

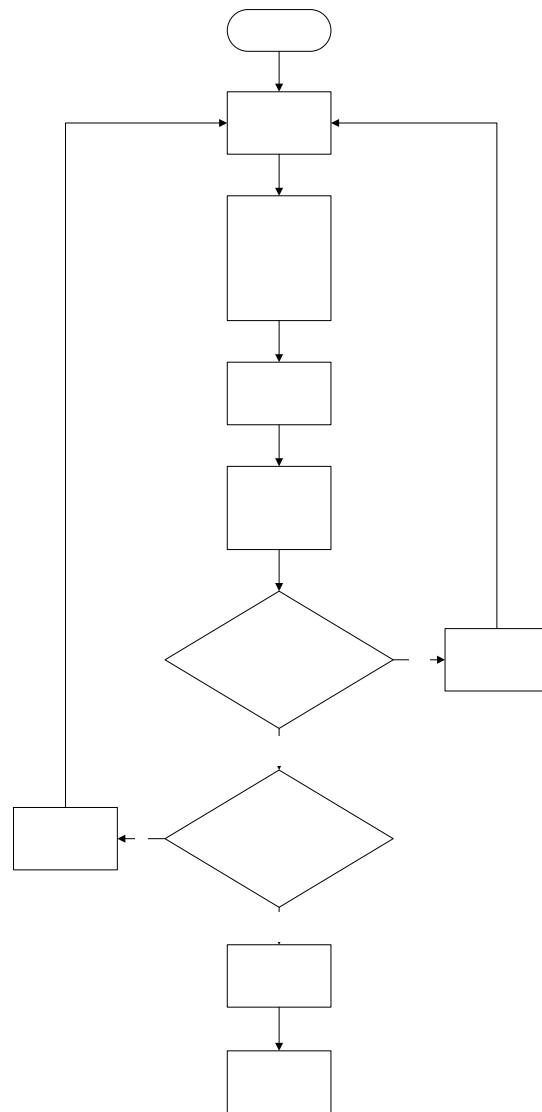
applet

Map.class

แสดงหน้าตาของ home.jsp

หน้าสมัครสมาชิกใหม่ (register.jsp)

เป็นที่ให้สมาชิกทำการกรอกข้อมูลเพื่อนำข้อมูลไปลงในฐานข้อมูลในตารางที่ชื่อ member

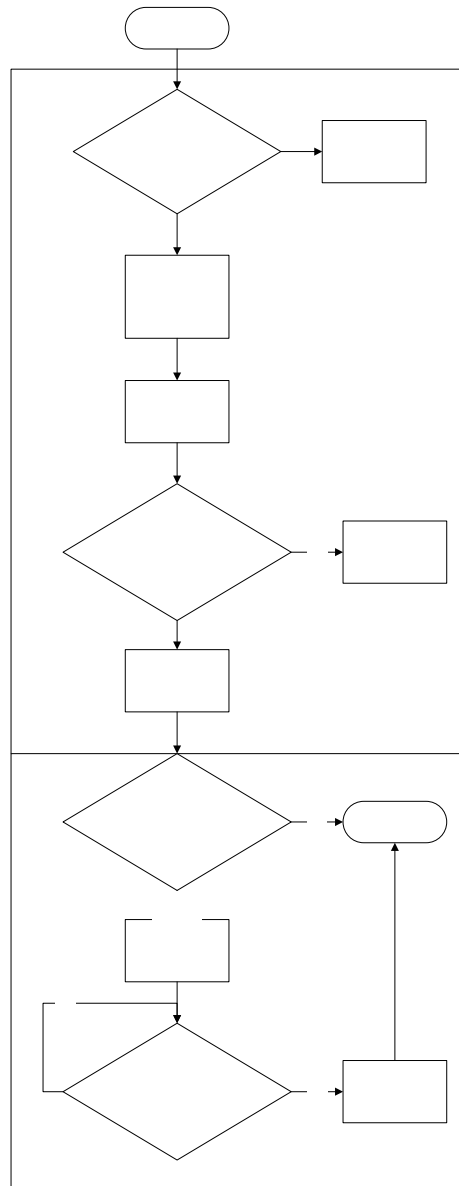


แสดงโฟลว์ชาร์ตของ register.jsp

ในการตรวจสอบความถูกต้องของอินพุตนั้นจะตรวจสอบว่า ชื่อ, อีเมล, รหัสผ่านทั้ง 2 ครั้ง และชื่อล็อกอิน ต้องอยู่ในรูปแบบที่ถูกต้อง คือ ต้องมีการป้อนและไม่ประกอบด้วยอักขระพิเศษ รวมทั้งตรวจสอบว่าจุด start และ destination ที่เป็นคู่เดียวกันนั้นต้องไม่ใช่จุดเดียวกัน และเมื่อมีการกรอกอินพุตของใคร่แล้ว อินพุตอื่นๆจะต้องมีการกรอกด้วย

หน้าแก้ไขข้อมูลส่วนตัวของสมาชิก (edit.jsp และ edit_cont.jsp)

เป็นหน้าสำหรับสมาชิกเท่านั้น ไว้ทำการแก้ไขข้อมูลส่วนตัวของตัวเองที่เคยให้ไว้ในฐานข้อมูล จากขั้นตอนการสมัครสมาชิก



แสดงโฟลว์ชาร์ตของ *edit.jsp* และ *edit_cont.jsp*

สำหรับการตรวจสอบอินพุตทุกอย่างจะเหมือนกันกับ *register.jsp*

หน้าล็อกอินหรือหน้าเข้าสู่ระบบสมาชิก (login.jsp)

เพื่อให้สมาชิกทำการล็อกอินเพื่อเข้าสู่ระบบ โดยการใช้พิสูจน์ตน (Authentication) แบบ username,password ซึ่งเมื่อสมาชิกผ่านการล็อกอินแล้วจะมีความสามารถในการเข้าสู่เว็บบอร์ดได้ และในการใช้งานระบบจะเพิ่มความสะดวกสบายมากขึ้นโดยการใช้ข้อมูลของสมาชิกเองในฐานะข้อมูลที่เคยให้ไว้กับระบบ

ซึ่งขั้นตอนนี้จะใช้เทคนิคของตัวแปรเซสชัน (session parameter) คือ จะใช้หมายเลขเซสชันของ http connection เป็นเหมือนกับหมายเลขตัวแทนในการติดต่อกับเซิร์ฟเวอร์ แล้วเก็บข้อมูลในฝั่งเซิร์ฟเวอร์สำหรับผู้ที่ผ่านการล็อกอินอย่างถูกต้องโดยใช้หมายเลขเซสชันเป็นตัวบ่งบอกว่าต้องเก็บที่ตัวแปรใด

ดังนั้นในการตรวจสอบสิทธิของสมาชิกจะใช้ตัวแปรเซสชันที่เก็บในฝั่งเซิร์ฟเวอร์เป็นตัวตรวจสอบ (ผู้ที่ผ่านการล็อกอินจะมีตัวแปร ผู้ที่ไม่ผ่านจะไม่มี)

ซึ่งโค้ดส่วนนี้จะเป็นดังนี้

```
if (session.getAttribute("auth") == null)
{
    response.sendRedirect("login.jsp");
    return;
}
```

หน้าล็อกเอาต์ออกจากระบบ (logout.jsp)

เป็นการลบตัวแปรเซสชันออกจากฝั่งเซิร์ฟเวอร์ทั้งหมด โดยใช้หมายเลขเซสชันเป็นตัวบ่งบอก เช่นเดียวกันกับขั้นตอนล็อกอิน

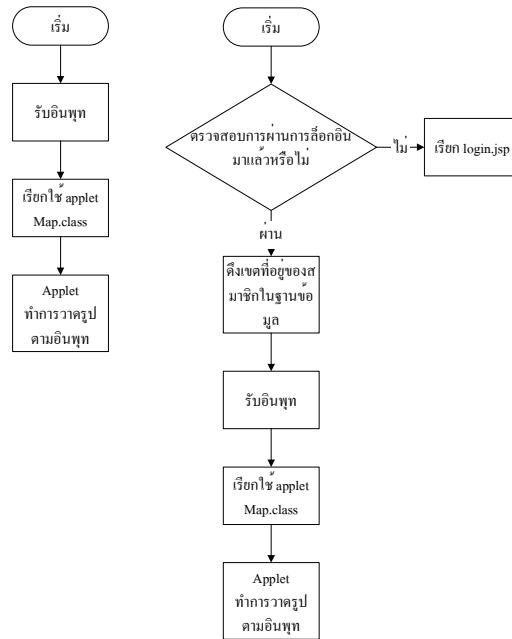
และสำหรับการที่สมาชิกที่ผ่านการล็อกอินแล้ว แต่ไม่ได้ทำการติดต่อกับระบบเป็นเวลานาน (20 นาที) จะทำให้เซสชันนั้นปิดตัวลงไป จะเสมือนกับว่าสมาชิกได้ทำการล็อกเอาต์ออกไปแล้ว

ซึ่งส่วนนี้จะใช้คำสั่ง

```
<%
    session.invalidate();
    response.sendRedirect("home.jsp");
%>
```

หน้าแสดงแผนที่และรายละเอียดของเส้นทางจราจร (map.jsp และ member_map.jsp)

จะแสดงแผนที่พร้อมด้วยสภาพการจราจรให้กับผู้ใช้ดู โดยต้องให้ผู้ใช้ทำการเลือกเขตที่ต้องการจะดูก่อน แต่สำหรับสมาชิกจะแสดงแผนที่ของเขตที่ตรงกับในฟิลด์ district ในตาราง member ในฐานะข้อมูลเลข โดยแผนที่นั้นจะใช้ applet เป็นตัววาด (Map.class) และสำหรับ applet เพื่อนำเมาส์ไปวางไว้บนตำแหน่งที่เป็นโหนด ก็จะแสดงรายละเอียดของโหนดนั้นๆออกมา



แสดงโฟลว์ชาร์ตของ map.jsp และ member_map.jsp

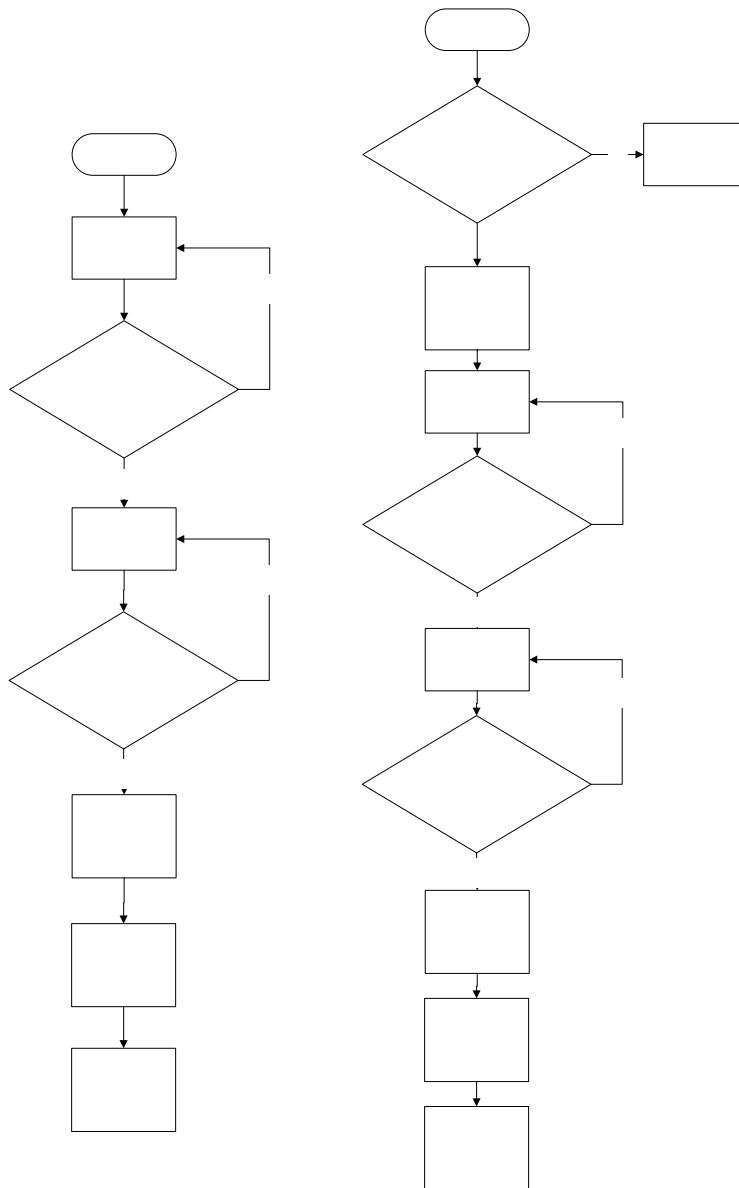
หน้าค้นหาเส้นทางจราจร (routing.jsp และ member_routing.jsp)

ให้ผู้ใช้ได้ทำการค้นหาบริเวณที่ต้องการก่อน โดยประเภทของบริเวณจะเป็นได้คือ ชื่อถนน, ชื่อโหนด และสถานที่ เช่น ถ้าทำการค้นหาคำว่า “ราช” ผลที่ได้จากการค้นหาก็คือ ราชประสงค์ ราชดำริ ราชดำเนิน เป็นต้น และผลการค้นหาจะแสดงออกมาเป็น radio box ให้ทำการเลือกว่าจะเลือกจากจุดเริ่มต้นใดไปยังจุดสิ้นสุดใด แต่สำหรับสมาชิกจะแสดงจุดเริ่มต้นและสิ้นสุดที่มีอยู่ในฐานข้อมูลเลยโดยไม่ต้องผ่านขั้นตอนการค้นหา

ซึ่งขั้นตอนต่อไปก็จะทำการเปลี่ยนจากจุดหรือบริเวณที่รับเข้ามาเป็นโหนดทั้งหมด เช่น หากเลือกเป็นชื่อถนนจะได้โหนด 2 โหนดด้วยกัน คือ แยกที่ถนนนั้นๆสิ้นสุด โดยที่การเปลี่ยนจากจุดหรือบริเวณเป็นโหนดนั้นจะใช้ข้อมูลในตาราง matching ในฐานข้อมูล (เปลี่ยนจากฟิลด์ word ไปเป็นฟิลด์ inter1 และ inter2) ดังนั้นจะเห็นได้ว่าจากจุดหรือบริเวณ 2 จุด (คือต้นทางและปลายทาง) จะได้เป็นโหนดทั้งหมด 4 โหนด

การจับคู่โหนดนั้นก็จับคู่ได้ทั้งหมด 4 แบบด้วยกัน แต่สำหรับส่วนประมวลผลนั้นจะทำงานได้แค่ทีละคู่เท่านั้น ดังนั้นจึงต้องส่งให้แก่ส่วนประมวลผลทีละคู่ และจะได้ผลการค้นหา 4 ผลด้วยกัน จากนั้นจึงตรวจสอบว่าเส้นทางใดมีผลตรงกับความต้องการของผู้ใช้มากที่สุด จึงเลือกผลนั้นมาทำการแสดงเพียงผลเดียว

ผู้ใช้สามารถเลือกรูปแบบการค้นหาได้ทั้งสิ้น 3 รูปแบบ คือ ระยะทางที่สั้นที่สุด ค่าใช้จ่ายที่ใช้ในการเดินทางน้อยที่สุด และระยะเวลาในการเดินทางน้อยที่สุด โดยทั้ง 3 แบบนี้สามารถนำมาใช้รวมกันอย่างไรก็ได้



แสดงโฟลว์ชาร์ตของ routing.jsp และ member_routing.jsp

สำหรับขั้นตอนการตรวจสอบอินพุตครั้งแรกนั้นจะดูว่าผู้ใช้ได้พิมพ์อินพุตเข้ามาหรือไม่ ถ้าพิมพ์เข้ามาก็จะตัดช่องว่างด้านหน้าและหลังของค่าที่รับเข้ามาก่อน แล้วค่อยนำมาค้นหาในฐานข้อมูล สำหรับการตรวจสอบอินพุตส่วนที่หลังนั้นจะดูว่าค่าจาก radio box ที่รับเข้ามา คือจุดเริ่มต้นและสิ้นสุดเป็นจุดเดียวกันหรือไม่ ถ้าเป็นจุดเดียวกันต้องทำการเลือกใหม่

หน้าแสดงรายงานอุบัติเหตุและสิ่งกีดขวางการจราจร (report.jsp และ member_report.jsp)

ให้ผู้ใช้ทำการค้นหาบริเวณที่ต้องการดูรายงานอุบัติเหตุหรือสิ่งกีดขวางการจราจรก่อน โดยบริเวณนั้นต้องเป็นได้ทั้งชื่อเขตและชื่อนาน คือถ้าเลือกค้นหาแบบชื่อเขต เมื่อทำการค้นหาแล้วผลที่จากการค้นหาจะเป็นรายชื่อ

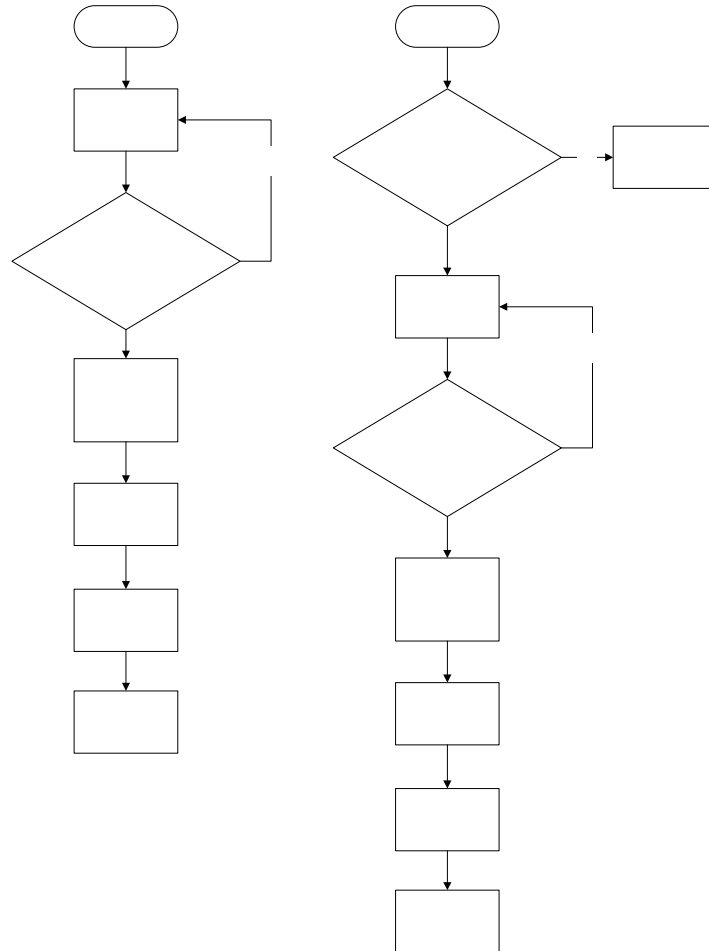
ตรวจสอบ
อินพุต

รั

แต่

ถนนที่อยู่ในเขตนั้นๆและมีรายงานเข้ามาทั้งหมด แต่ถ้าเลือกเป็นชื่อถนน ผลที่ได้จากการค้นหาจะเป็นเพียงถนนที่ค้นเจอเท่านั้น

จากนั้นผลของการค้นหาจะแสดงออกมาเป็น check box เพื่อให้เลือกแสดงรายงานเฉพาะบริเวณที่ต้องการเท่านั้น



แสดงไฟล์ชาร์ตของ report.jsp และ member_report.jsp

หน้าแสดงรายงานสดวิทยุออนไลน์ จส.100 (radio.jsp และ member_radio.jsp)

เรียกใช้ plugin ของ Windows Media Player ใน Microsoft Internet Explorer มาทำการเล่น URL ของสตรีมที่เป็นรายงานสดของวิทยุ จส.100

โดยมีคำสั่งดังนี้คือ

```

<Embed SRC="mms://live.virginradiothailand.com/105.5"
Width="337"
Height="70"
AutoStart="false"

```

ตรวจสอบ
อินพุต

```

TYPE="application/x-mplayer2"
pluginspage="http://www.microsoft.com/Windows/MediaPlayer/"
Name="objMediaPlayer"
autoSize="false"
showdisplay="false"
displaySize="0"
center="true"
ShowControls="true"
ShowStatusBar="true"
AnimationAtStart="false" border="2"></embed>

```



แสดงplugin ของ Windows Media Player

หน้าต้อนรับเข้าสู่ระบบสมาชิกหลังทำการล็อกอิน (member_section.jsp)

เป็นหน้าต้อนรับสมาชิกผู้ผ่านการล็อกอินเข้าสู่ระบบมาอย่างถูกต้องแล้วเท่านั้น

หน้าแสดงรายชื่อกระทู้ทั้งหมดของเว็บบอร์ด (webboard.jsp)

แสดงหัวข้อกระทู้ทั้งหมดที่มี โดยจะแบ่งแสดงเป็นหน้าๆ หน้าละ 10 หัวข้อกระทู้ ดังนั้นจึงต้องคำนวณก่อนว่าจะมีจำนวนหน้าทั้งหมดกี่หน้า

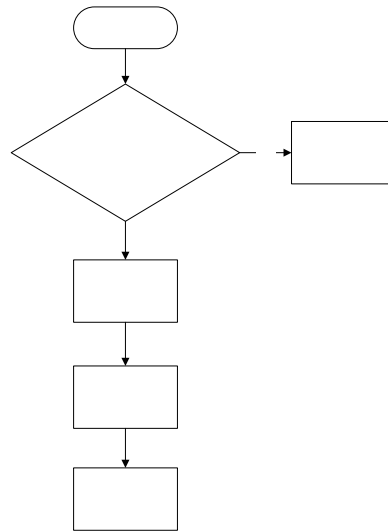
```
int total_page = (int)Math.ceil(((double)total_row/(double)row_page);
```

จากนั้นทำการคำนวณต่อว่า จะต้องดึงข้อมูลจากฐานข้อมูลเข้ามาตั้งแต่กระทู้ที่เท่าไรถึงเท่าไร โดยการหาจุดเริ่มต้นการ

```
start = (screen - 1)*row_page
```

screen คือ หมายเลขหน้าที่ต้องการเปิดดู จากนั้นก็ทำการดึงข้อมูลออกมาจำนวน 10 กระทู้

ทั้งนี้หมายเลขก่อนหลังของกระทู้จะต้องเรียงตามวันเวลาที่สร้างกระทู้คือ สร้างก่อนจะมีหมายเลขต่ำกว่า



แสดงไฟล์ชาร์ตของ webboard.jsp

หน้าสร้างกระทู้ใหม่หรือหน้าโพสต์ (addnew.jsp)

สำหรับสมาชิกเพื่อทำการสร้างกระทู้ใหม่ โดยจะมีการเก็บวันที่โพสต์ หมายเลข IP address ของเครื่องที่ใช้ติดต่อเข้ามาด้วย ส่วนข้อมูลอื่นๆจะเก็บจากค่าในฐานข้อมูลของสมาชิกคนนั้นๆ เช่น อีเมลแอดเดรส

โดยที่เพจนี้อาจมีการเรียกเมธอด (Method) ชื่อ filter_topic และ filter_comment ด้วย โดยที่ภายในเมธอดนี้จะมีรายละเอียดดังนี้

```

public static String filter_topic(String input)
{
    StringBuffer temp = new StringBuffer(input.length());
    char c;
    int j=1;
    for (int i=0;i<input.length();i++)
    {
        c = input.charAt(i);
        if (c == '<') temp.append("&lt;");
        else if (c == '>') temp.append("&gt;");
        else if (c == "\\") temp.append("&#039;");
        else if (c == "\"") temp.append("&quot;");
        else if (c == '&') temp.append("&amp;");
        else if (c == "\\") temp.append("&#92;");
        else temp.append(c);
    }
}
  
```

```

        if (j == 30)
        {
            temp.append("<br>");
            j = 0;
        }
        j++;
    }
    return(temp.toString());
}

```

```

public static String filter_comment(String input)
{
    StringBuffer temp = new StringBuffer(input.length());
    char c;
    int j=1;
    for (int i=0;i<input.length();i++)
    {
        c = input.charAt(i);
        if (c == '<') temp.append("&lt;");
        else if (c == '>') temp.append("&gt;");
        else if (c == "\n") temp.append("&#039;");
        else if (c == "\"") temp.append("&quot;");
        else if (c == '&') temp.append("&amp;");
        else if (c == "\\") temp.append("&#92;");
        else temp.append(c);

        if (j == 50)
        {
            temp.append("<br>");
            j = 0;
        }
        if (c == "\n")
            j = 0;
    }
}

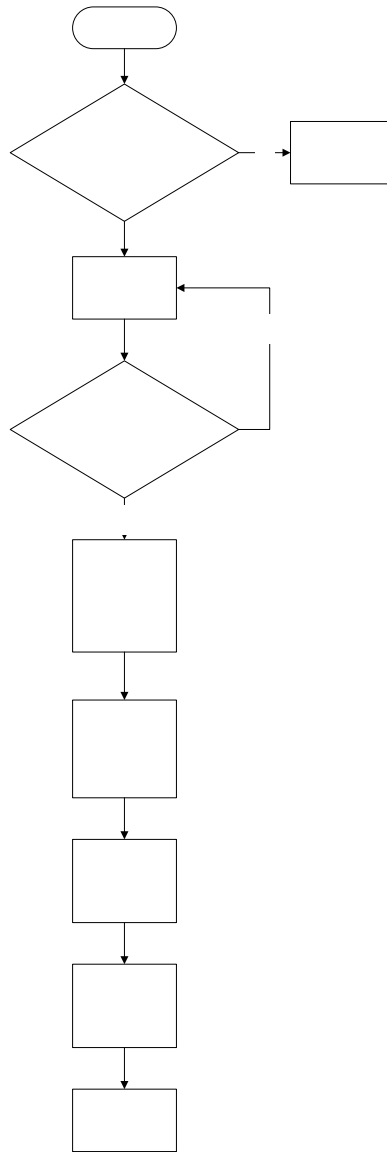
```

```
        j++;  
    }  
    return(temp.toString());  
}
```

คือมีหน้าที่ในการแปลงข้อความที่รับมาจากสมาชิก แล้วทำการเปลี่ยนจากอักขระพิเศษที่จะทำให้เกิดปัญหาหรือเป็นอันตรายแก่เซิร์ฟเวอร์ได้เป็นอักขระอื่นๆดังนี้คือ

<	เป็น	<
>	เป็น	>
\	เป็น	'
"	เป็น	"
&	เป็น	&
\\	เป็น	\

และจะมีการตัดค่าความยาวของตัวอักษร คือ เมื่อข้อความมีความยาวเกิน 50 ตัวอักษรและ 30 ตัวอักษร สำหรับ filter_comment และ filter_topic ตามลำดับ โดยไม่มีการเว้นวรรคหรือขึ้นบรรทัดใหม่ จะมีการแทรก “
” ลงไปในข้อความนั้นๆด้วย เพื่อให้รูปแบบตอนแสดงผลของตัวอักษรมีรูปแบบที่ไม่เป็นปัญหา



แสดงโฟลว์ชาร์ตของ *addnew.jsp*

ในการตรวจสอบอินพุตนั้นจะแค่ดูว่าสมาชิกได้มีการพิมพ์ชื่อกระทู้และรายละเอียดลงมาหรือไม่

หน้าแสดงและตอบกระทู้ (view.jsp)

เมื่อสมาชิกทำการคลิกเมาส์ที่ชื่อกระทู้ภายในหน้า webboard.jsp ก็จะทำการเปิดเข้ามาดูว่าภายในกระทู้มีอะไรอยู่บ้าง และสามารถทำการตอบกระทู้นั้นๆได้

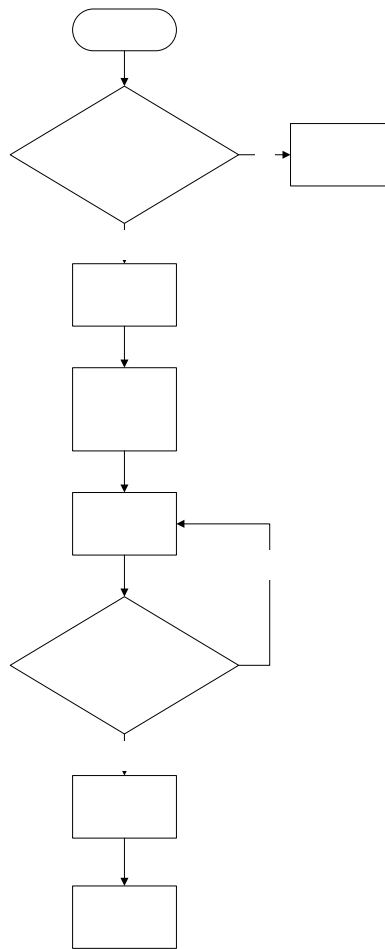
เพื่อนี้จะทำการเรียกเมธอด filter_comment เช่นเดียวกันกับ addnew.jsp และเรียกเมธอด newline เพิ่มขึ้นมาซึ่งมีการทำงานดังนี้

```

public static String newline(String input)
{
    StringBuffer temp = new StringBuffer();
    StringTokenizer st = new StringTokenizer(input, "\n");
    while(st.hasMoreTokens())
        temp.append(st.nextToken() + "<br>");
    return(temp.toString());
}

```

คือเนื่องจากการรับข้อความเข้ามาทาง textfield ของเพจ HTML เมื่อเกิดการเว้นบรรทัดขึ้นจะแทนด้วยอักขระ "\n" ซึ่งการที่เราจะนำข้อความนี้ไปแสดงบนเพจ HTML จะต้องทำการแปลงให้เป็น
 ก่อน ซึ่งเป็นคำสั่งเว้นบรรทัดจริงๆ ดังนั้นจึงต้องเรียกเมธอดนี้ก่อนการแสดงผล



แสดงไฟล์ชาร์ตของ view.jsp

หน้าแสดงความผิดพลาดของเซิร์ฟเวอร์ (error.jsp)

หน้านี้จะถูกแสดงขึ้นมาเมื่อเกิดความผิดพลาดของเว็บเซิร์ฟเวอร์ขึ้น โดยเว็บเซิร์ฟเวอร์จะเรียกขึ้นมาเองเลข เพื่อเป็นการแจ้งข้อผิดพลาด และป้อนค่าตัวแปร (parameter) ต่างๆที่ไม่ควรจะให้ผู้ใช้งานเห็นด้วยคำสั่ง

```
<%@ errorPage="error.jsp"%>
```

ไฟล์เก็บส่วนหัวและแถบเมนูด้านข้าง (header.jsp, member_header.jsp และ member_left.jsp)

เพื่อการแก้สิ่งที่อาจเกิดขึ้นภายหลังด้วยความง่ายและสะดวก จึงได้ทำการรวมลิงค์ที่ต้องมีในแต่ละเพจที่เหมือนกันไว้ในไฟล์เดียว ด้วยคำสั่ง

```
<jsp:include page="header.jsp"/>
```

```
<jsp:include page="member_header.jsp"/>
```

```
<jsp:include page="member_left.jsp"/>
```

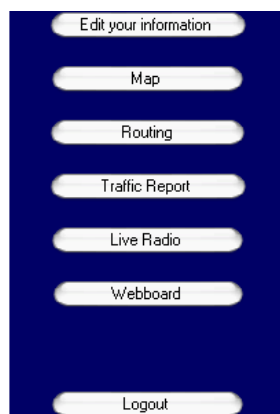
โดยที่ไฟล์ header.jsp จะถูกเรียกใช้โดยเพจที่ผู้ใช้ไม่ได้ผ่านการล็อกอินเข้าสู่ระบบ ส่วน member_header.jsp และ member_left.jsp จะถูกเรียกใช้ในเพจที่ผู้ใช้เป็นสมาชิกและผ่านการล็อกอินแล้ว



หน้าตาของ member_header.jsp



หน้าตาของ header.jsp



หน้าตาของ member_left.jsp

โปรแกรม Java Applet ที่ใช้แสดงแผนที่ (Map.class)

ใช้ในการแสดงและคอยกำกับการแสดงของส่วนที่เป็นรูปแผนที่โดยเฉพาะ คือ จะทำการเลื่อนรูปภาพไปได้ 8 ทิศทาง เลือกรูปภาพที่จะนำมาแสดง ทำการวาดรูปหรือเพิ่มจุดที่เป็นโหนดลงในแผนที่ และจากนั้นก็ทำการแสดงรายละเอียดของบริเวณที่ผู้ใช้ต้องการ

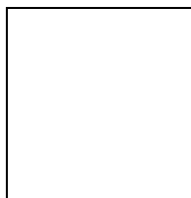
Applet นี้ถูกแบ่งแยกการทำงานออกเป็นหลายๆส่วนซึ่งทำงานไม่เหมือนกัน แต่เกี่ยวข้องกัน โดยแบ่งเป็นเมธอดที่ ซึ่งมีเมธอดที่สำคัญดังนี้คือ

- CountXY ใช้ในการนับจำนวนของโหนดที่มีอยู่ในภาพที่กำลังแสดงอยู่ เพื่อให้โปรแกรมสามารถทำงานได้ถูกต้อง โดยจะเป็นตัวกำหนดจำนวนการวนลูป (For loop) เพื่อการวาดโหนดในรูปนั้นๆ
- getX ใช้ในการดึงค่าลำดับแกน X ของโหนดแต่ละ โหนดในภาพจากฐานข้อมูลที่กำลังแสดงอยู่ เพื่อใช้ในการวาดจุดโหนด
- getY ใช้ในการดึงค่าลำดับแกน Y ของโหนดแต่ละ โหนดในภาพจากฐานข้อมูลที่กำลังแสดงอยู่ เพื่อใช้ในการวาดจุดโหนด
- shownode ใช้ในการแสดงรายละเอียดของโหนดนั้นๆ เมื่อนำเมาส์ไปวางไว้ยังตำแหน่งที่เป็นโหนด
- getPic ใช้ในการติดต่อส่วนที่เป็น JSP คือ เมื่อผู้ใช้ทำการเลือกเขตที่จะแสดงรูปแผนที่ เมธอดนี้จะทำการหาจากฐานข้อมูลว่า จะต้องแสดงรูปใด
- กลุ่มเมธอดที่ใช้แสดงสถานะเมื่อทำการเลื่อนเมาส์
- และกลุ่มเมธอดที่ใช้ในการเปลี่ยนภาพที่แสดงไปในทิศทางต่างๆ

และสำหรับชื่อไฟล์ที่เป็นรูปภาพแผนที่ที่จะต้องทำการเก็บนำลักษณะคู่อันดับ XY เช่น “e10044600n1346000s4.jpg” โดยที่ตัวเลขชุดแรกที่อยู่ทางด้านซ้ายมือนั้นคือค่าของแกน X และถัดมาทางขวามือคือค่าแกน Y โดยที่เมื่อทำการเปลี่ยนภาพที่จะแสดงนั้นค่าเหล่านี้จะบวกหรือลบด้วย 2400 (เมื่อต้องการภาพที่อยู่ด้านบนจะบวกแกน Y ด้วย 2400 เช่นกันทางด้านล่างจะลบด้วย 2400 และสำหรับเมื่อต้องการจะเลื่อนไปทางด้านซ้ายจะลบค่าแกน X ด้วย 2400 เช่นกันทางด้านขวาจะบวกด้วย 2400)

แต่คู่อันดับของโหนดแต่ละ โหนดนั้นจะต่างกัน คือ แต่ละภาพจะไม่ขึ้นต่อกัน เช่น ภาพ ก. จะมีคู่อันดับ (x,y) ตั้งแต่ (0,0) ถึง (600,600) และภาพ ข. ก็จะมีตั้งแต่ (0,0) ถึง (600,600) เหมือนกัน

(0,0)



(600,600)

รูปแบบคู่อันดับ XY ของโหนด

ขั้นตอนการทำงานส่วนแสดงผลของโทรศัพท์เคลื่อนที่

โครงสร้างของระบบค้นหาเส้นทางจราจรบนโทรศัพท์เคลื่อนที่แบ่งได้เป็น 2 กลุ่ม คือ กลุ่มของไฟล์ที่ประกอบกันเป็นแอปพลิเคชันใช้งานบนโทรศัพท์เคลื่อนที่ และกลุ่มของไฟล์สำหรับประมวลผลบนฝั่งเซิร์ฟเวอร์

ไฟล์สำหรับแอปพลิเคชัน

แอปพลิเคชันจะถูกเขียนขึ้นและใช้ในฝั่งไคลเอนต์โดยภาษาที่ใช้หลักการของ Object Orientation ซึ่งมีการเขียนคลาสออกเป็นหลายคลาส เมธอดและตัวแปรต่างๆ จะถูกเรียกใช้ผ่านออบเจกต์ที่ได้ประกาศไว้ การออกแบบคลาสจะออกแบบตามประเภทของหน้าการแสดงผล เช่น คลาส UserScreen สำหรับแสดงหน้าการเลือกประเภทของผู้ใช้บริการ เป็นต้น ทั้งนี้เพื่อแยกหน้าที่การทำงานในแต่ละหน้าอย่างชัดเจน ซึ่งประกอบด้วยไฟล์ทั้งหมด ดังนี้

ไฟล์ TrafficReport.java

เป็นไฟล์สำหรับคลาส TrafficReport ซึ่งเป็นคลาสหลักที่มีการประกาศเมธอดต่างๆ ที่ใช้สำหรับประมวลผลและการแสดงผลออกทางจอภาพของโทรศัพท์เคลื่อนที่ โดยการแสดงผลจะใช้การประกาศออบเจกต์ของคลาสอื่นๆ ร่วมกับคลาส Display ด้วย เมธอดต่างๆ ในคลาส TrafficReport มีดังนี้

- เมธอด **TrafficReport()** เป็นคอนสตรัคเตอร์ของคลาสที่ไม่มีการใช้งาน
- เมธอด **StartApp()** เป็นเมธอดของ MIDlet ใช้เพื่อจองทรัพยากรของระบบ และถือเป็นจุดเริ่มของโปรแกรมด้วย เมื่อ MIDlet เริ่มทำงานเมธอดนี้จะถูกเรียกขึ้นมา
- เมธอด **PauseApp()** เป็นเมธอดของ MIDlet ที่บอก MIDlet ให้พักการทำงานชั่วคราวและคืนทรัพยากรสู่ระบบ
- เมธอด **DestroyApp()** เป็นเมธอดของ MIDlet เพื่อจบการทำงานของ MIDlet และคืนทรัพยากรทั้งหมดกลับสู่ระบบ
- เมธอด **goUser()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส UserScreen เพื่อแสดงหน้าการเลือกประเภทผู้ใช้บริการ ออกทางจอภาพ
- เมธอด **goLogin()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส LoginScreen เพื่อแสดงหน้าการตรวจสอบความเป็นสมาชิกของผู้ใช้งานระบบ ออกทางจอภาพ
- เมธอด **goMember()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส MemberScreen เพื่อแสดงหน้าการเลือกประเภทของบริการสำหรับสมาชิก ออกทางจอภาพ โดยในคลาสนี้จะมีการติดต่อกับเซิร์ฟเวอร์เพื่อร้องขอข้อมูลของสมาชิก
- เมธอด **goNonmember()** เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส NonmemberScreen เพื่อแสดงหน้าการเลือกประเภทของบริการสำหรับผู้ที่ไม่ใช่สมาชิก ออกทางจอภาพ

- เมธอด `goSearchsource()` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `SearchsourceScreen` เพื่อแสดงหน้าการเลือกและค้นหาเส้นทาง ออกทางจอภาพ
- เมธอด `goSearchdest()` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `SearchdestScreen` เพื่อแสดงหน้าการเลือกและค้นหาปลายทาง ออกทางจอภาพ
- เมธอด `goCost()` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `CostScreen` เพื่อแสดงหน้าการเลือกปัจจัยในการค้นหาเส้นทาง ออกทางจอภาพ
- เมธอด `goReport()` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `ReportareaScreen` เพื่อแสดงหน้าการเลือกและค้นหาเขตรายงาน ออกทางจอภาพ
- เมธอด `goRange()` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `ReportrangeScreen` เพื่อแสดงหน้าการเลือกพื้นที่ของเขตรายงาน ออกทางจอภาพ
- เมธอด `goResponse(String input)` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `ResponseScreen` เพื่อแสดงผลลัพธ์เนื่องจากการค้นหาเส้นทางและการรายงานสภาพจราจรในพื้นที่เขตออกทางจอภาพ
- เมธอด `goEdituser()` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `EdituserScreen` เพื่อแสดงหน้าการแก้ไขข้อมูลของสมาชิก ออกทางจอภาพ
- เมธอด `goEditdistrict()` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `EditdistrictScreen` เพื่อแสดงหน้าการแก้ไขข้อมูลเขตรายงาน ออกทางจอภาพ
- เมธอด `goEditok()` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `Alert` เพื่อแสดงหน้าของการแก้ไขข้อมูลสมาชิกสำเร็จโดยจะแสดงออกมาชั่วระยะเวลาหนึ่งก่อนที่จะเข้าสู่หน้าประเภทของบริการ
- เมธอด `goError(String flag)` เป็นเมธอดที่มีการประกาศออบเจกต์ของคลาส `Alert` เพื่อแสดงหน้าความผิดพลาดอันเนื่องมาจากบริการต่างๆ ออกทางจอภาพ โดยในหน้าของความผิดพลาดจะแสดงในช่วงระยะเวลาหนึ่งก่อนที่จะเข้าสู่หน้าประเภทของบริการ
- เมธอด `goConnect(String input)` เป็นเมธอดที่ใช้สำหรับส่งข้อมูลร้องขอและรับข้อมูลตอบรับต่างๆ ระหว่างผู้ใช้งานโทรศัพท์เคลื่อนที่และเครื่องเซิร์ฟเวอร์ผู้ให้บริการ

ไฟล์ `UserScreen.java`

เป็นไฟล์สำหรับคลาส `UserScreen` มีหน้าที่เพื่อแสดงประเภทของผู้ใช้บริการ คือ ผู้ใช้บริการที่เป็นสมาชิกและผู้ใช้บริการทั่วไป สืบทอดมาจากคลาส `List` ที่แสดงเป็นลำดับรายการให้ผู้ใช้งานเลือกได้ ประกอบด้วยตัวแปรและเมธอดต่างๆ ที่สำคัญ ดังนี้

- ตัวแปร `userStatus` เป็นตัวแปรชนิด บูลีน ใช้สำหรับเก็บสถานะของผู้ใช้ ปกติเมื่อโปรแกรมทำงานค่า `userstatus` จะถูกกำหนดให้เป็น `false` ซึ่งจะหมายถึงผู้ใช้งานในขณะนี้เป็นผู้ใช้งานทั่วไป และ `userstatus` จะถูกกำหนดเป็น `true` สำหรับสมาชิก

- เมธอด **UserScreen(TrafficReport midlet)** เป็นคอนสตรักเตอร์สำหรับสร้างองค์ประกอบของการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ลำดับรายการเลือก และ ปุ่ม Select
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส UserScreen

ไฟล์ LoginScreen.java

เป็นไฟล์สำหรับคลาส LoginScreen ที่แสดงหน้าของการเข้าสู่ระบบของสมาชิก สืบทอดคุณสมบัติมาจากคลาส Form โดยคลาส LoginScreen จะมีตัวแปรและเมธอดที่สำคัญ ดังนี้

- ตัวแปร **usr_pass** เป็นตัวแปรชนิดสตริง ใช้สำหรับเก็บข้อมูลชื่อและรหัสผ่านของผู้ใช้ เพื่อใช้ในการร้องขอข้อมูลสมาชิกที่เครื่องเซิร์ฟเวอร์
- ตัวแปร **usr** เป็นตัวแปรชนิดออบเจกต์ของคลาส UserScreen สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- เมธอด **LoginScreen(TrafficReport midlet)** เป็นคอนสตรักเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลชื่อผู้ใช้, ฟิลด์ข้อมูลรหัสผ่าน ปุ่ม Select และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส LoginScreen

ไฟล์ MemberScreen.java

เป็นไฟล์สำหรับคลาส MemberScreen ที่แสดงหน้าบริการสำหรับสมาชิก ที่สืบทอดมาจากคลาส List ได้แก่ บริการค้นหาเส้นทาง, บริการรายงานสภาพจราจร และบริการการแก้ไขข้อมูลสมาชิก ซึ่งเป็นบริการเฉพาะสำหรับสมาชิกเท่านั้น คลาส MemberScreen จะมีตัวแปรและเมธอดที่สำคัญดังนี้

- ตัวแปร **infoList** เป็นตัวแปรชนิดอาร์เรย์ แบบสตริง ขนาด 14 ช่องที่ใช้สำหรับเก็บข้อมูลต่างๆ ของสมาชิก ได้แก่ ชื่อสมาชิก, รหัสผ่าน, ชื่อจริง, อีเมลล์, ที่อยู่, ชื่อเขตรายงาน, ผู้ให้บริการโทรศัพท์, เบอร์โทรศัพท์, ชื่อผู้ใช้โทรศัพท์, รหัสผ่านโทรศัพท์, ต้นทาง1, ต้นทาง2, ปลายทาง1 และ ปลายทาง2 สำหรับข้อมูลเขตรายงาน, ต้นทางและปลายทาง จะช่วยเพิ่มความสะดวกให้กับสมาชิกในกรณีที่สมาชิกต้องการค้นหาเส้นทางจราจรที่ไปเป็นประจำ หรือต้องการทราบรายงานสภาพจราจรในเขตพื้นที่รายงานเดิม สมาชิกไม่ต้องเสียเวลากับการค้นหารายชื่อ
- ตัวแปร **source** เป็นตัวแปรชนิดออบเจกต์ของคลาส SearchsourceScreen สำหรับเรียกใช้งาน ตัวแปรหรือ เมธอดในคลาส

- ตัวแปร **dest** เป็นตัวแปรชนิดออบเจกต์ของคลาส SearchdestScreen สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- ตัวแปร **report** เป็นตัวแปรชนิดออบเจกต์ของคลาส ReportareaScreen สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- เมธอด **MemberScreen(TrafficReport midlet, String input)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ List ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, รายการบริการต่างๆ และปุ่ม Select รวมถึงการนำผลลัพธ์ของข้อมูลสมาชิกที่ได้จากเครื่องเซิร์ฟเวอร์มาเก็บลงในอาร์เรย์
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส MemberScreen

ไฟล์ NonmemberScreen.java

เป็นไฟล์สำหรับคลาส NonmemberScreen ที่แสดงหน้าบริการสำหรับผู้ใช้งานทั่วไป สืบทอดมาจากคลาส List ได้แก่ บริการค้นหาเส้นทาง, บริการรายงานสภาพจราจร NonmemberScreen จะมีเมธอดที่สำคัญดังนี้

- เมธอด **NonmemberScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ List ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, รายการบริการสำหรับผู้ใช้งานทั่วไป
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส NonmemberScreen

ไฟล์ SearchsourceScreen.java

เป็นไฟล์สำหรับคลาส SearchsourceScreen ที่แสดงหน้าบริการค้นหาเส้นทางจราจร สืบทอดมาจากคลาส Form เพื่อวางคอมโพเนนต์ชนิด Item ได้หลากหลาย SearchsourceScreen จะมีตัวแปรและ เมธอดที่สำคัญดังนี้

- ตัวแปร **sourceList** เป็นตัวแปรชนิดอาร์เรย์ แบบสตริง ขนาด 50 ช่อง ใช้สำหรับเก็บข้อมูลรายชื่อเส้นทางที่ค้นหาเพื่อเตรียมมาวางเป็นลำดับรายการ
- ตัวแปร **routeArray** เป็นตัวแปรชนิดอาร์เรย์ แบบสตริง ขนาด 5 ช่อง ใช้สำหรับเก็บข้อมูลเส้นทาง, ปลายทาง และปัจจัยในการค้นหาเส้นทาง เพื่อใช้ในการค้นหาเส้นทางจากต้นทางไปยังปลายทางโดยคำนึงถึงปัจจัยที่เลือก โดยจะส่งเป็นข้อมูลร้องขอไปยังเครื่องเซิร์ฟเวอร์
- ตัวแปร **usr** เป็นตัวแปรชนิดออบเจกต์ของคลาส UserScreen สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- เมธอด **SearchsourceScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลสำหรับใช้ค้นหารายการชื่อเส้นทาง, คอมโพเนนต์ ประเภทแสดงลำดับรายการ, ปุ่ม Select, ปุ่ม Search และปุ่ม Back

- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแฟกเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส SearchsourceScreen

ไฟล์ SearchdestScreen.java

เป็นไฟล์สำหรับคลาส SearchdestScreen ที่แสดงหน้าบริการค้นหาปลายทางจราจร สืบทอดมาจากคลาส Form เพื่อวางคอมโพเนนต์ชนิด Item ได้หลากหลาย SearchdestScreen จะมีตัวแปรและเมธอดที่สำคัญดังนี้

- ตัวแปร **destList** เป็นตัวแปรชนิดอาร์เรย์ แบบสตริง ขนาด 50 ช่องใช้สำหรับเก็บข้อมูลรายชื่อปลายทางที่ค้นหาเพื่อเตรียมมาวางเป็นลำดับรายการ
- ตัวแปร **source** เป็นตัวแปรชนิดออบเจกต์ของคลาส SearchsourceScreen สำหรับเรียกใช้งานตัวแปรหรือเมธอดในคลาส
- เมธอด **SearchdestScreen(TrafficReport midlet)** เป็นคอนสตรักเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลสำหรับใช้ค้นหาชื่อปลายทาง, คอมโพเนนต์ประเภทลำดับรายการ สำหรับแสดงรายการชื่อปลายทาง, ปุ่ม Select, ปุ่ม Search และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแฟกเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส SearchdestScreen

ไฟล์ CostScreen.java

เป็นไฟล์สำหรับคลาส CostScreen ที่แสดงหน้ารายการเลือกปัจจัยในการค้นหาเส้นทาง ได้แก่ ค่าใช้จ่าย, เวลา และระยะทาง ตามความต้องการในการค้นหาเส้นทางที่แตกต่างกันของผู้ใช้บริการ สืบทอดมาจากคลาส List โดยมี ตัวแปรและเมธอดที่สำคัญดังนี้

- ตัวแปร **source** เป็นตัวแปรชนิดออบเจกต์ของคลาส SearchsourceScreen สำหรับเรียกใช้งานตัวแปรหรือเมธอดในคลาส
- เมธอด **CostScreen(TrafficReport midlet)** เป็นคอนสตรักเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, คอมโพเนนต์ประเภท Choice Group สำหรับแสดงรายการปัจจัยที่มีผลต่อการค้นหาเส้นทาง, ปุ่ม Route! และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแฟกเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส CostScreen

ไฟล์ ReportareaScreen.java

เป็นไฟล์สำหรับคลาส ReportareaScreen ที่ใช้สำหรับการแสดงผลหน้ารายการเพื่อให้ผู้ใช้สามารถเลือกเขตที่ต้องการทราบสภาพจราจรในขณะนั้น สืบทอดมาจากคลาส Form มีตัวแปรและเมธอดที่สำคัญ ดังนี้

- ตัวแปร **areaList** เป็นตัวแปรอาร์เรย์แบบสตริงขนาด 50 ช่อง สำหรับเก็บรายการเขตที่ค้นหาได้เพื่อนำไปแสดงผล โดยข้อมูลที่ได้จะได้อมาจากการร้องขอที่เครื่องเซิร์ฟเวอร์
- ตัวแปร **usr** เป็นตัวแปรชนิดออบเจกต์ของคลาส **UserScreen** สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- ตัวแปร **mem** เป็นตัวแปรชนิดออบเจกต์ของคลาส **MemberScreen** สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- ตัวแปร **range** เป็นตัวแปรชนิดออบเจกต์ของคลาส **ReportrangeScreen** สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- เมธอด **ReportareaScreen(TrafficReport midlet)** เป็นคอนสตรักเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, คอมโพเนนต์ Choice Group แสดงลำดับรายการเขต, ฟิลด์ข้อมูลรหัสผ่าน ปุ่ม Select, ปุ่ม Search และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส **ReportareaScreen**

ไฟล์ **ReportrangeScreen.java**

เป็นไฟล์สำหรับคลาส **ReportrangeScreen** ที่ใช้สำหรับการแสดงผลหน้ารายการพื้นที่ที่มีการรายงานสภาพจราจรในขณะนั้น โดยจะแสดงในลักษณะของลำดับรายการ มีคุณสมบัติสืบทอดมาจากคลาส **List** มีตัวแปรและเมธอดที่สำคัญดังนี้

- ตัวแปร **rangeList** เป็นตัวแปรอาร์เรย์แบบสตริงขนาด 50 ช่อง สำหรับเก็บรายการพื้นที่ที่ค้นหาได้เพื่อนำไปแสดงผล โดยข้อมูลที่ได้จะได้อมาจากการร้องขอที่เครื่องเซิร์ฟเวอร์
- เมธอด **ReportrangeScreen(TrafficReport midlet, String input)** เป็นคอนสตรักเตอร์สำหรับสร้างองค์ประกอบของ **List** ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลผลลัพธ์, ปุ่ม Report และปุ่ม Back รวมถึงขั้นตอนการนำข้อมูลผลลัพธ์มาใส่ในฟิลด์ข้อมูล
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส **ReportrangeScreen**

ไฟล์ **ResponseScreen.java**

เป็นไฟล์สำหรับคลาส **ResponseScreen** ที่ใช้ในการแสดงผลหน้าผลลัพธ์เนื่องจากการค้นหาเส้นทางหรือการรายงานสภาพการจราจร โดยจะรับข้อมูลเข้ามาและนำข้อมูลไปแสดงผลออกทางจอภาพ คลาส **ResponseScreen** จะมีคุณสมบัติสืบทอดมาจากคลาส **Form** โดยจะมีตัวแปรและเมธอด ต่างๆที่สำคัญดังนี้

- ตัวแปร **usr** เป็นตัวแปรชนิดออบเจกต์ของคลาส **UserScreen** สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- ตัวแปร **report** เป็นตัวแปรชนิดออบเจกต์ของคลาส **ReportrangeScreen** สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- เมธอด **ResponseScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลผลลัพธ์, ปุ่ม Service และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ **j2me** อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส **ResponseScreen**

ไฟล์ **EdituserScreen.java**

เป็นไฟล์สำหรับคลาส **EdituserScreen** ใช้สำหรับสมาชิกในกรณีที่มีสมาชิกเรียกใช้บริการการแก้ไขข้อมูล โดยคลาสจะเป็นส่วนแสดงผลหน้าฟิลด์ข้อมูลต่างๆ เกี่ยวกับสมาชิกทั้งหมด เช่น ข้อมูลชื่อ, ข้อมูลที่อยู่ และข้อมูลชื่ออีเมล ทั้งนี้เพื่อให้สมาชิกสามารถแก้ไขข้อมูลบนโทรศัพท์เคลื่อนที่ได้ มีตัวแปรและเมธอดต่างๆ ที่สำคัญ ดังนี้

- ตัวแปร **pmet** เป็นตัวแปรอาร์เรย์ ชนิดสตริงขนาด 13 ช่อง ใช้สำหรับเก็บข้อมูลใหม่ที่มีสมาชิกแก้ไขแล้ว ยกเว้นข้อมูลเขตรายงานที่สมาชิกจะต้องเลือกในหน้าแก้ไขข้อมูลเขตรายงานซึ่งแยกออกไปต่างหาก ก่อนที่จะส่งไปเปลี่ยนแปลงที่ฐานข้อมูลบนเครื่องเซิร์ฟเวอร์
- ตัวแปร **mem** เป็นตัวแปรชนิดออบเจกต์ของคลาส **MemberScreen** สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- ตัวแปร **distr** เป็นตัวแปรชนิดออบเจกต์ของคลาส **EditdistrictScreen** สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- เมธอด **EdituserScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลค้นหารายการเขต, คอมโพเนนต์ลำดับรายการ ปุ่ม Next และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ **j2me** อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส **EdituserScreen**

ไฟล์ **EditdistrictScreen.java**

เป็นไฟล์สำหรับคลาส **EditdistrictScreen** ใช้สำหรับสมาชิกในกรณีที่มีสมาชิกเรียกใช้บริการการแก้ไขข้อมูล โดยคลาสจะเป็นส่วนแสดงผลหน้าฟิลด์ข้อมูลเฉพาะข้อมูลเขตรายงานเท่านั้น เนื่องจากจะมีลำดับรายการเขตให้สมาชิกสามารถเลือกได้ ซึ่งประหยัดเวลาในการพิมพ์ชื่อให้ถูกต้อง คลาส **EditdistrictScreen** สืบทอดมาจากคลาส **Form** โดยมีเมธอดต่างๆ ที่สำคัญ ดังนี้

- ตัวแปร **mem** เป็นตัวแปรชนิดออบเจกต์ของคลาส MemberScreen สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- ตัวแปร **log** เป็นตัวแปรชนิดออบเจกต์ของคลาส LoginScreen สำหรับเรียกใช้งานตัวแปรหรือ เมธอดในคลาส
- เมธอด **EditdistrictScreen(TrafficReport midlet)** เป็นคอนสตรัคเตอร์สำหรับสร้างองค์ประกอบของ Form ในการแสดงผล ได้แก่ หัวข้อของหน้าแสดงผล, ฟิลด์ข้อมูลค้นหารายการเขตรายงาน, คอมโพเนนต์ลำดับรายการเขต, ปุ่ม Edit, ปุ่ม Search และปุ่ม Back
- เมธอด **commandAction(Command c, Displayable d)** เป็นเมธอดที่มีอยู่ในแพ็คเกจของ j2me อยู่แล้วใช้ตรวจสอบเหตุการณ์ของการกดปุ่ม ในคลาส EditdistrictScreen

ไฟล์บนฝั่งเซิร์ฟเวอร์

ที่ฝั่งเซิร์ฟเวอร์จะเขียนเป็นไฟล์ JSP ทั้งหมด 9 ไฟล์ โดยแต่ละไฟล์จะทำหน้าที่ในการติดต่อไปยังฐานข้อมูลด้วยคำสั่งในการค้นหาข้อมูล ที่แตกต่างกัน จากนั้นจะนำค่าข้อมูลที่ได้ส่งกลับไปฝั่งไคลเอนต์เพื่อใช้ประมวลผลต่อไป ไฟล์ต่างๆ มีดังนี้

ไฟล์ Server_Authen.jsp

เป็นไฟล์สำหรับการตรวจสอบความเป็นสมาชิกของผู้ใช้งาน ทำหน้าที่ในการรับข้อมูล ชื่อสมาชิกและรหัสผ่าน และใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลใน ฟิลด์ชื่อสมาชิกและฟิลด์รหัสผ่านตรงกับข้อมูลที่รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะมีค่ารีเทิร์นกลับมาเป็นเลข 1 เซิร์ฟเวอร์จะส่งค่ากลับไปเป็นสตริงว่า “ok” ถ้าค้นหาไม่พบ ผลลัพธ์ที่ได้จะมีค่ารีเทิร์นกลับมาเป็นเลขอื่นๆ ที่ไม่ใช่เลข 1 และเซิร์ฟเวอร์จะส่งค่ากลับไปเป็นสตริงว่า “no”

```
ResultSet myresult = stmt.executeQuery("SELECT count(*) as num FROM member WHERE
usr='"+requirer+"' and pass='"+reqpass+"'"); //set sql command to search username and password field
while(myresult.next()) {
    if(myresult.getInt("num")==1)
        out.println("ok"); //search complete return ok
    else
        out.println("no"); //search fail return no
}
```

ไฟล์ Server_EditSearch.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลต่างๆ ที่เกี่ยวกับสมาชิก เพื่อใช้ในบริการประเภทต่างๆ เช่น บริการค้นหาเส้นทาง จะมีข้อมูลต้นทางและปลายทางปรากฏให้สมาชิกได้เลือกอยู่แล้ว เป็นการประหยัดเวลาในการเลือกรายการต้นทางและปลายทาง ไฟล์จะทำหน้าที่ในการรับข้อมูล ชื่อสมาชิกและรหัสผ่าน จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลใน ฟیلด์ชื่อสมาชิกและฟیلด์รหัสผ่านตรงกับข้อมูลที่รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของข้อมูลสมาชิกส่งกลับไปฝั่งไคลเอนต์เพื่อนำไปใช้งานต่อไป ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลสมาชิก

```
ResultSet myresult = stmt.executeQuery("SELECT * FROM member WHERE usr='"+requirer+"' and  
pass='"+reqpass+"'"); //set sql command to search member information  
if(myresult != null) {  
    while(myresult.next()) {  
        output = myresult.getString("usr") +',' + //get information in each field and-  
myresult.getString("pass")+',' + //concatenate with the original output  
        myresult.getString("name")+',' +  
        myresult.getString("email")+',' +  
        myresult.getString("district")+',' +  
        myresult.getString("address")+',' +  
        myresult.getString("mobile")+',' +  
        myresult.getString("provider")+',' +  
        myresult.getString("mo_usr")+',' +  
        myresult.getString("mo_pass")+',' +  
        myresult.getString("start1")+',' +  
        myresult.getString("dest1")+',' +  
        myresult.getString("start2")+',' +  
        myresult.getString("dest2")+',' +$";  
    }  
    out.println(output); //return to client  
}
```

ไฟล์ Server_RouteSearch.jsp

เป็นไฟล์สำหรับการค้นหารายการข้อมูล ต้นทางหรือปลายทาง ในกรณีที่ผู้ใช้บริการไม่ทราบชื่อของต้นทางหรือปลายทางที่ถูกต้อง ไฟล์จะทำหน้าที่ในการรับข้อมูลที่เป็นคีย์เวิร์ดในการค้นหา จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหารายการต้นทางหรือปลายทางที่มีตัวอักษรในข้อมูลตรงกับคีย์เวิร์ด ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็น

สตริงของข้อมูลรายการค้นหาทางหรือปลายทาง ส่งกลับไปที่ฝั่งไคลเอนต์เพื่อนำไปใช้ผู้ใช้งานได้เลือก ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลค้นหาและปลายทาง

```
ResultSet myresult = stmt.executeQuery("SELECT * FROM matching WHERE word like  
%" + reqsearch + "%"); //set sql command to search anywhere matched keyword  
while(myresult.next()){  
    output = output + myresult.getString("word") + ','; //concatenate result with original output  
}  
output = output + '$';  
out.println(output); //return to client
```

ไฟล์ Server_RouteJunction.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลแยกของถนนหรือเรียกว่า โหนด เพื่อใช้ในการค้นหาเส้นทาง หลังจากที่ได้ค้นหาทางหรือปลายทางแล้วระบบจะติดต่อมายังไฟล์ โดยไฟล์จะทำหน้าที่ในการรับข้อมูลชื่อค้นหาทางหรือปลายทาง จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลในฟิลด์ชื่อค้นหาทางหรือปลายทางตรงกับข้อมูลที่รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของข้อมูลแยกหรือโหนดของค้นหาทางหรือปลายทาง ส่งกลับไปที่ฝั่งไคลเอนต์เพื่อนำไปใช้งานต่อไป ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลสมาชิก

```
ResultSet myresult = stmt.executeQuery("SELECT * FROM matching WHERE word='" + reqjunc + "'");  
while(myresult.next()){ //set sql command to get junction nearly places  
    output = output + myresult.getString("inter1") + ',' + myresult.getString("inter2") + ',';  
}  
//get junction and concatenate with the original output  
output = output + '$'; //return to client  
out.println(output);
```

ไฟล์ Server_RouteResponse.jsp

เป็นไฟล์สำหรับการส่งข้อมูลไปยังโปรแกรมค้นหาเส้นทาง ไฟล์จะทำหน้าที่ในการรับข้อมูล ชื่อค้นหาทางและปลายทาง ได้แก่ ค้นหาทาง1, ค้นหาทาง2, ปลายทาง1, ปลายทาง2 และ ปัจจัยการค้นหาเส้นทางจากนั้นใช้จาวาบิน ในการติดต่อไปยังโปรแกรมค้นหาเส้นทาง เมื่อการค้นหาเสร็จสิ้นระบบจะได้เส้นทางที่ดีที่สุดส่งกลับไปยังฝั่งไคลเอนต์ ซึ่งเป็นเส้นทางที่ผู้ใช้สามารถเดินทางได้อย่างมีประสิทธิภาพ

ไฟล์ Server_AreaSearch.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลเขตพื้นที่รายงาน เพื่อใช้ในบริการ รายงานสภาพจราจรและบริการแก้ไขข้อมูลสมาชิก ซึ่งต้องมีการค้นหารายชื่อเขตที่ต้องการ ไฟล์จะทำหน้าที่ในการรับข้อมูลคีย์เวิร์ดเข้ามา จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาในแถวข้อมูลที่มีตัวอักษรตรงกับคีย์เวิร์ด ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของข้อมูลรายการเขตส่งกลับไปฝั่งไคลเอนต์เพื่อนำไปเลือกพื้นที่รายงานต่อไป ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลเขต

```
ResultSet myresult = stmt.executeQuery("SELECT distr_name FROM district WHERE distr_name like
'%" + reqarea + "%'");           //set sql command to search area matched keyword
while(myresult.next()){
    output = output + myresult.getString("distr_name") + ',';    //concatenate areas with the-
}                                                                    //original output
output = output + "$";           //return to client
```

ไฟล์ Server_RangeSearch.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลพื้นที่ในเขตรายงาน จะทำงานหลังจากที่ผู้ใช้งานได้เลือกเขตแล้ว พื้นที่รายงานจะเป็นส่วนหนึ่งของเขตรายงาน เนื่องจากเขต 1 เขตมีขนาดใหญ่ โดยไฟล์จะทำหน้าที่ในการรับข้อมูลชื่อเขตรายงาน จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลในฟิลด์ชื่อเขตตรงกับข้อมูลที่รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของรายการพื้นที่ต่างๆ ในเขตที่การรายงานสภาพจราจร และจะส่งกลับไปฝั่งไคลเอนต์ให้ผู้ใช้ได้เลือกต่อไป ถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลรายการพื้นที่

```
ResultSet myresult = stmt.executeQuery("SELECT * FROM report WHERE district='" + reqarea + "'");
while(myresult.next()){           // set sql command to search area
    output = output + myresult.getString("street") + ',';    //get the ranges in the area
}
output = output + "$";           //return to client
```

ไฟล์ Server_ReportResponse.jsp

เป็นไฟล์สำหรับการค้นหาข้อมูลรายละเอียดสภาพการจราจรในเขตพื้นที่ที่ได้ออกไว้ ไฟล์จะทำหน้าที่ในการรับข้อมูลชื่อพื้นที่รายงานเข้ามา จากนั้นใช้คำสั่งค้นหาในฐานข้อมูล ค้นหาแถวของข้อมูลที่มีข้อมูลในฟิลด์ชื่อพื้นที่ตรงกับข้อมูลที่รับเข้ามา ถ้าค้นหาเจอ ผลลัพธ์ที่ได้จะเป็นสตริงของข้อมูลรายละเอียดการรายงานสภาพการจราจรในพื้นที่บริเวณนั้น เช่น เกิดอุบัติเหตุ หรือ มีสิ่งกีดขวาง เป็นต้น ข้อมูลจะถูกส่งกลับไปฝั่งไคลเอนต์เพื่อนำไปแสดงผลออกทางจอภาพ และถ้าค้นหาไม่พบผลลัพธ์จะเป็นอย่างอื่นที่ไม่ใช่ข้อมูลรายละเอียดการรายงานสภาพจราจร

```

ResultSet myresult = stmt.executeQuery("SELECT * FROM report WHERE street='"+reqstr+"'");
while(myresult.next()){           // set sql command to get report by range in the area
    output = myresult.getString("date")+','+myresult.getString("comment");
}
                                //get report
output = output + ",";           //return to client

```

ไฟล์ Server_EditUpdate.jsp

เป็นไฟล์สำหรับการแก้ไขข้อมูลในฐานข้อมูลของสมาชิก โดยไฟล์จะทำหน้าที่ในการรับข้อมูล ทั้งหมดของสมาชิก จากนั้นใช้คำสั่งอัปเดตฐานข้อมูล อัปเดตแถวของข้อมูลที่มีข้อมูลในฟิลด์ชื่อสมาชิกตรงกับข้อมูลที่รับเข้ามา ถ้าการอัปเดตสำเร็จ ผลลัพธ์ที่ได้จะเป็นเลข 1 ถ้าค้นหาไม่พบผลลัพธ์จะเป็นเลขอื่นที่ไม่ใช่เลข 1 และจะส่งกลับไปฝั่งไคลเอนต์เพื่อใช้งานต่อไป

```

String sql = "UPDATE member SET
usr='"+requiser+"',pass='"+reqpass+"',name='"+reqname+"',email='"+reqemail+"',district='"+reqdistr+"',
address='"+reqaddr+"',mobile='"+reqmob+"',provider='"+reqprov+"',mo_usr='"+reqmousr+"',
mo_pass='"+reqmopss+"' WHERE usr='"+requiser+"'"; //set sql to go update member information
int result = stmt.executeUpdate(sql);
if(result == 1)                                //update succeeded
    out.println("ok");
else                                            //update not succeeded
    out.println("no");

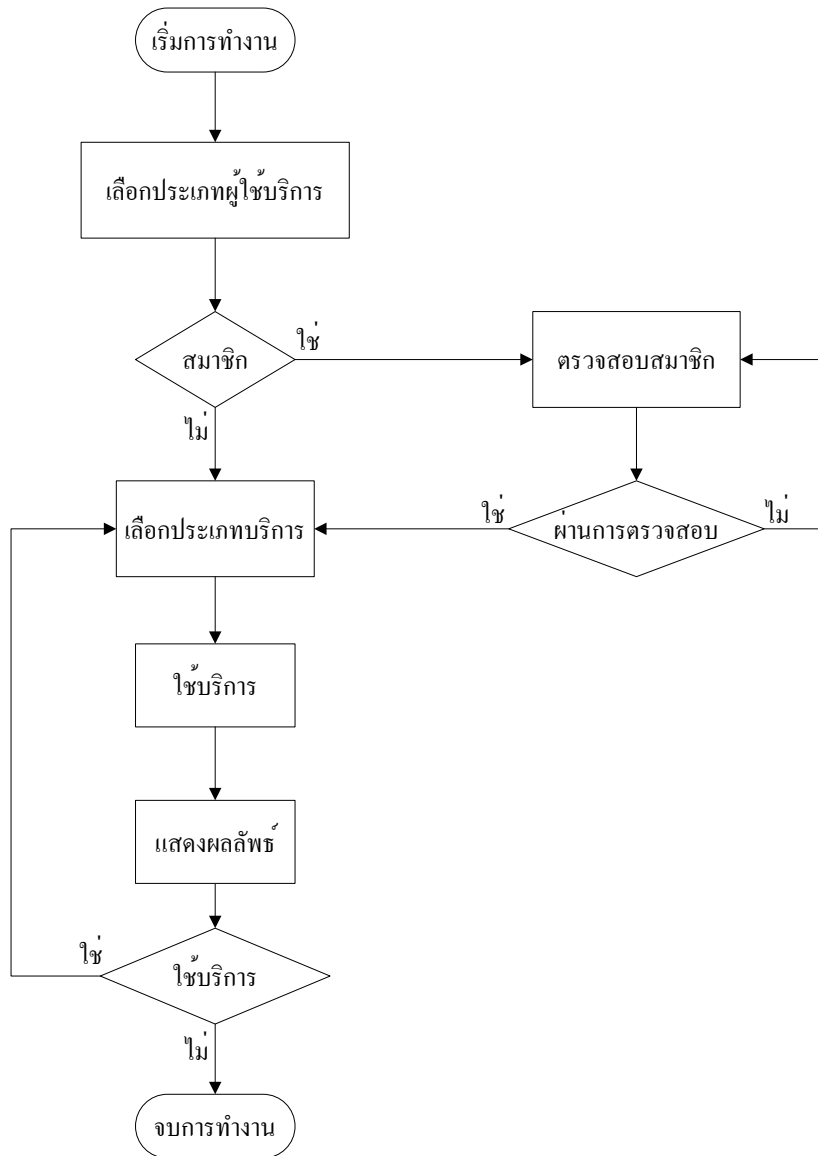
```

การทำงานของโปรแกรมบนโทรศัพท์เคลื่อนที่

การทำงานของแอปพลิเคชันจะกล่าวถึง ในลักษณะของกิจกรรมที่ผู้ใช้งานกระทำกับตัวระบบ โดยมี เซิร์ฟเวอร์และฐานข้อมูลเป็นส่วนประกอบในการทำงานด้วย ซึ่งสามารถแยกได้ 4 ประเภท ได้แก่ การทำงานของ แอปพลิเคชันโดยรวม, การทำงานของบริการค้นหาเส้นทางจราจร, การทำงานของบริการรายงานสภาพการจราจร และการทำงานของบริการแก้ไขข้อมูลสมาชิก

การทำงานของแอปพลิเคชันโดยรวม

ขั้นตอนการทำงานของแอปพลิเคชันโดยรวมจะเป็นไปตามโฟลว์ชาร์ตของการทำงาน ดังรูปซึ่งมีขั้นตอน ดังนี้



แสดงฟลัวร์ชาร์ต การทำงานของระบบโดยรวม

- ในขั้นตอนแรกจะต้องเลือก ประเภทของผู้ใช้งาน ได้แก่ สมาชิก หรือ ผู้ใช้งานทั่วไป เพราะสมาชิกจะแตกต่างกับผู้ใช้งานทั่วไปตรงที่มีข้อมูลเส้นทาง, ปลายทาง และข้อมูลเขตรายงานที่เป็นประโยชน์แก่สมาชิก ในบริการค้นหาเส้นทาง, บริการรายงานสภาพการจราจร และบริการแก้ไขข้อมูลสมาชิก
- เมื่อผู้ใช้บริการรันแอปพลิเคชัน ระบบจะเข้าสู่คลาส TrafficReport โดยไปเรียกใช้เมธอด startApp() ซึ่งเป็นจุดเริ่มต้นการทำงานของ MIDlet
- ภายในเมธอด startApp() โปรแกรมจะมีการเรียกใช้เมธอด goUser()

- ภายในเมธอด goUser() จะมีการประกาศออบเจ็กต์ User คลาส UserScreen เรียกใช้เมธอดคอนสตรัคเตอร์ UserScreen(TrafficReport midlet) เพื่อแสดงผลหน้าเลือกประเภทของผู้ใช้งาน โดยพารามิเตอร์ที่ส่งเข้าไปคือออบเจ็กต์ ของคลาสหลักที่ใช้สำหรับแสดงค่าคอมโพเนนต์ต่างๆออกทางหน้าจอ ณ ปัจจุบัน

```
UserScreen(TrafficReport midlet) {
    super("USER",List.IMPLICIT); // page title
    this.midlet = midlet;
    append("Member",null); // list choice 1
    append("Non Memeber",null); // list choice 2
    Select = new Command("Select",Command.SCREEN,0); // only one command in this page
    addCommand(Select);
    setCommandListener(this);
}
```

- เมื่อผู้ใช้บริการเลือกประเภทของผู้ใช้แล้วกดปุ่ม Next เมธอด commandAction(Command c,Displayable d) ในคลาส UserScreen จะทำหน้าที่ดักจับค่า c ซึ่งก็คือ เหตุการณ์ที่ผู้ใช้งานกดปุ่ม และตรวจสอบ
- หากพบว่าค่า c เท่ากับ Next จะตรวจสอบค่าดัชนีลำดับ (getSelectedIndex(n)) อีกครั้ง ถ้าค่าดัชนีลำดับ เท่ากับ 0 หมายความว่า ผู้ใช้บริการเลือกบริการแบบสมาชิก ระบบจะเรียกใช้งานเมธอด goLogin() แต่ถ้าหากค่าดัชนีลำดับเท่ากับ 1 ระบบจะกำหนดค่า userStatus เป็น false เพื่อบ่งบอกว่าผู้ใช้บริการเลือกบริการแบบผู้ใช้บริการทั่วไปและเรียกใช้งานเมธอด goNonmember()

```
public void commandAction(Command c,Displayable d) {
    int n;
    if(c == Select) { //detect command select
        n = getSelectedIndex();
        if(n == 0) // ok you are member then go to login first
            midlet.goLogin();
        if(n == 1) { //ok you are non member
            userStatus = false;
            midlet.goNonmember();
        }
    }
}
```

- ในขั้นตอนถัดไป ถ้าผู้ใช้บริการที่เป็นสมาชิก จะมีการพิสูจน์การมีตัวตนของสมาชิก โดยให้สมาชิกระบุชื่อสมาชิกและรหัสผ่าน ในขั้นตอนนี้ภายในเมธอด goLogin() จะประกาศออบเจกต์ Login คลาส LoginScreen และเรียกใช้งานเมธอดคอนสตรัคเตอร์ LoginScreen(TrafficReport midlet) เพื่อแสดงผลหน้าเลือกประเภทของผู้ใช้งาน โดยพารามิเตอร์ที่ส่งเข้าไปคือออบเจกต์ ของคลาสหลักที่ใช้สำหรับแสดงค่าคอมโพเนนต์ต่างๆออกทางหน้าจอ ณ ปัจจุบัน

```

LoginScreen(TrafficReport midlet) {
    super("LOGIN");
    this.midlet = midlet;
    userField = new TextField("Username:", "", 20, TextField.ANY);
    append(userField);
    passField = new TextField("Password:", "", 20, TextField.PASSWORD);
    append(passField);
    ItemStateListener listener = new ItemStateListener()
    {
        public void itemStateChanged(Item item)
        {
            temp0 = userField.getString();
            temp1 = passField.getString();
        }
    };
    Login = new Command("Login", Command.SCREEN, 0);
    addCommand(Login);
    Back = new Command("Back", Command.SCREEN, 0);
    addCommand(Back);
    setCommandListener(this);
    setItemStateListener(listener);
}

```

- เมื่อสมาชิกระบุชื่อและรหัสผ่านแล้วกดปุ่ม Login เมธอด commandAction(Command c, Displayable d) ในคลาส LoginScreen จะทำหน้าที่ดักจับค่า c ซึ่งก็คือ เหตุการณ์ที่ผู้ใช้งานกดปุ่ม และตรวจสอบ

```

public void commandAction(Command c, Displayable d) {

    String result = new String();

    String encode = new String();

    if(c == Login) {
        try {

            usr.userStatus = true;

            encode =

midlet.EncodeConnection("username="+temp0+"&password="+temp1);

            serverURL = "http://localhost:8080/Server_Authen.jsp?" + encode;
            result = midlet.goConnect(serverURL);
            result = result.trim();
            if(result.equals("ok")) {

                usr_pss = encode; // usr_pass store username and password again to
member get informations

                midlet.goMember();

            }

            else

                midlet.goError("log");

        }

        catch (IOException e) {

        }

    }

    if(c == Back)

        midlet.goUser();

}

```

- หากพบว่าค่า c เท่ากับ Login โปรแกรมจะมีการกำหนดค่า usr.userStatus เท่ากับ true เพื่อป้องกันว่าผู้ใช้มีสถานะเป็นสมาชิก และส่งข้อมูลไปยังเซิร์ฟเวอร์ที่ไฟล์ Server_Authen.jsp ผ่าน เมธอด goConnect(serverURL) โดยพารามิเตอร์ที่ส่งไปคือ สตริงของข้อมูลที่ใช้ในการตรวจสอบ หากพบว่าค่า c เท่ากับ Back โปรแกรมจะกลับไปสู่หน้าแสดงผลก่อนหน้านั้น
- ที่ไฟล์ Server_Authen.jsp จะรับค่าชื่อและรหัสผ่านสมาชิกมาเก็บยังตัวแปร requester และ reqpass ตามลำดับ จากนั้นจะทำการค้นหารายการที่ตรงกับตัวแปรในฐานข้อมูลด้วยคำสั่ง SQL

- ผลลัพธ์ที่ได้จากการค้นหาเมื่ออยู่ 2 ประเภท คือ หากค้นหาเจอไฟล์จะส่งค่าข้อมูลเป็น 1 กลับไปยังโทรศัพท์เคลื่อนที่ แต่หากค้นหาไม่พบ ไฟล์จะส่งค่าอื่นๆ ที่ไม่เท่ากับ 1 กลับไปแทน
- ที่แอปพลิเคชันจะได้รับผลลัพธ์ที่ส่งกลับมาจากไฟล์ Server_Authen.jsp แล้วตรวจสอบ หากพบว่ามีค่าเท่ากับ 1 ระบบจะเก็บค่าชื่อสมาชิกและรหัสผ่านในตัวแปร usr_pss สำหรับการเรียกข้อมูลของสมาชิกในครั้งถัดไปและเรียกใช้เมธอด goMember() แต่หากตรวจสอบพบว่าไม่เท่ากับ 1 หมายความว่าข้อมูลชื่อและรหัสผ่านผิดพลาด ระบบจะเรียกใช้เมธอด goError("log") โดยค่าที่ส่งเข้าไปหมายความว่าป็นข้อผิดพลาดที่เกิดขึ้นจากหน้าล็อกอิน

```
if(flag.equals("log")) {
    Err.setString("Username or Password is not correct!!!");
    LoginScreen errLog = new LoginScreen(this);
    Display.getDisplay(this).setCurrent(Err,errLog);
}
```

- ที่เมธอด goError(String flag) จะตรวจสอบแหล่งของข้อผิดพลาด ในที่นี้ เมธอดตรวจสอบพบว่าค่า flag มีค่าเท่ากับ "log" หมายความว่า เป็นข้อผิดพลาดจากหน้าล็อกอิน ระบบจะประกาศออบเจกต์ Err เรียกใช้งานตัวคอมโพเนนต์ Alert เพื่อแสดงข้อผิดพลาดให้ผู้ใช้งานทราบเป็นระยะเวลา 5 วินาที จากนั้นจะกลับไปสู่หน้าล็อกอินอีกครั้ง
- เมื่อสมาชิกผ่านการตรวจสอบ ระบบจะเข้าสู่หน้า MemberScreen โดยในเมธอด goMember() จะมีการส่งข้อมูลชื่อและรหัสผ่านสมาชิก โดยตัวแปร usr_pss ไปยังไฟล์ Server_EditSearch.jsp ผ่านเมธอด goConnect(String input)

```
public void goMember() {
    try{
        LoginScreen log = new LoginScreen(this);
        String result = new String();
        result = goConnect("http://localhost:8080/Server_EditSearch.jsp?" + log.usr_pss);
        result = result.substring(2,result.length()-1);
        MemberScreen Mem = new MemberScreen(this,result);
        Display.getDisplay(this).setCurrent(Mem);
    }
    catch(IOException e) {
```

```

    }
}

```

- ที่ไฟล์ Server_EditSearch.jsp จะรับข้อมูลชื่อและรหัสผ่านเข้ามาแล้วค้นหาในฐานข้อมูล เมื่อพบข้อมูลของสมาชิกแล้วไฟล์จะส่งสตรีมข้อมูลสมาชิกกลับมายังโทรศัพท์เคลื่อนที่
- ที่แอปพลิเคชันเมื่อรับข้อมูลเข้ามาแล้ว จะเรียกใช้เมธอดคอนสตรัคเตอร์ของคลาส MemberScreen เพื่อแสดงผลหน้าเลือกประเภทของบริการ ได้แก่ บริการค้นหาเส้นทางจราจร บริการรายงานสภาพจราจรและบริการแก้ไขข้อมูลสมาชิก โดยพารามิเตอร์ที่ส่งเข้าไปคืออินเจกต์ ของคลาสหลักที่ใช้สำหรับแสดงค่าคอมโพเนนต์ต่างๆออกทางหน้าจอ ณ ปัจจุบัน และข้อมูลของสมาชิก

```

MemberScreen(TrafficReport midlet, String input) {
    super("SERVICE",List.IMPLICIT);
    this.midlet = midlet;
    getinfoList(input);
    append("Search Route",null);
    append("Report Area",null);
    append("Edit Personal",null);
    Select = new Command("Select",Command.SCREEN,0);
    addCommand(Select);
    setCommandListener(this);
}

```

- ที่เมธอด MemberScreen(TrafficReport midlet,String input) จะมีการเรียกใช้ เมธอด getinfoList(String input) เพื่อเก็บข้อมูลลงในตัวแปรอาร์เรย์ infoList

```

public void getinfoList(String input) {
    for(int i=0;i<input.length();i++) {
        if(input.charAt(i) == ',') {
            if(input.charAt(i+1)=='$') {
                infoList[k] = input.substring(j,i);
                i = input.length();
            }
            else {

```

```

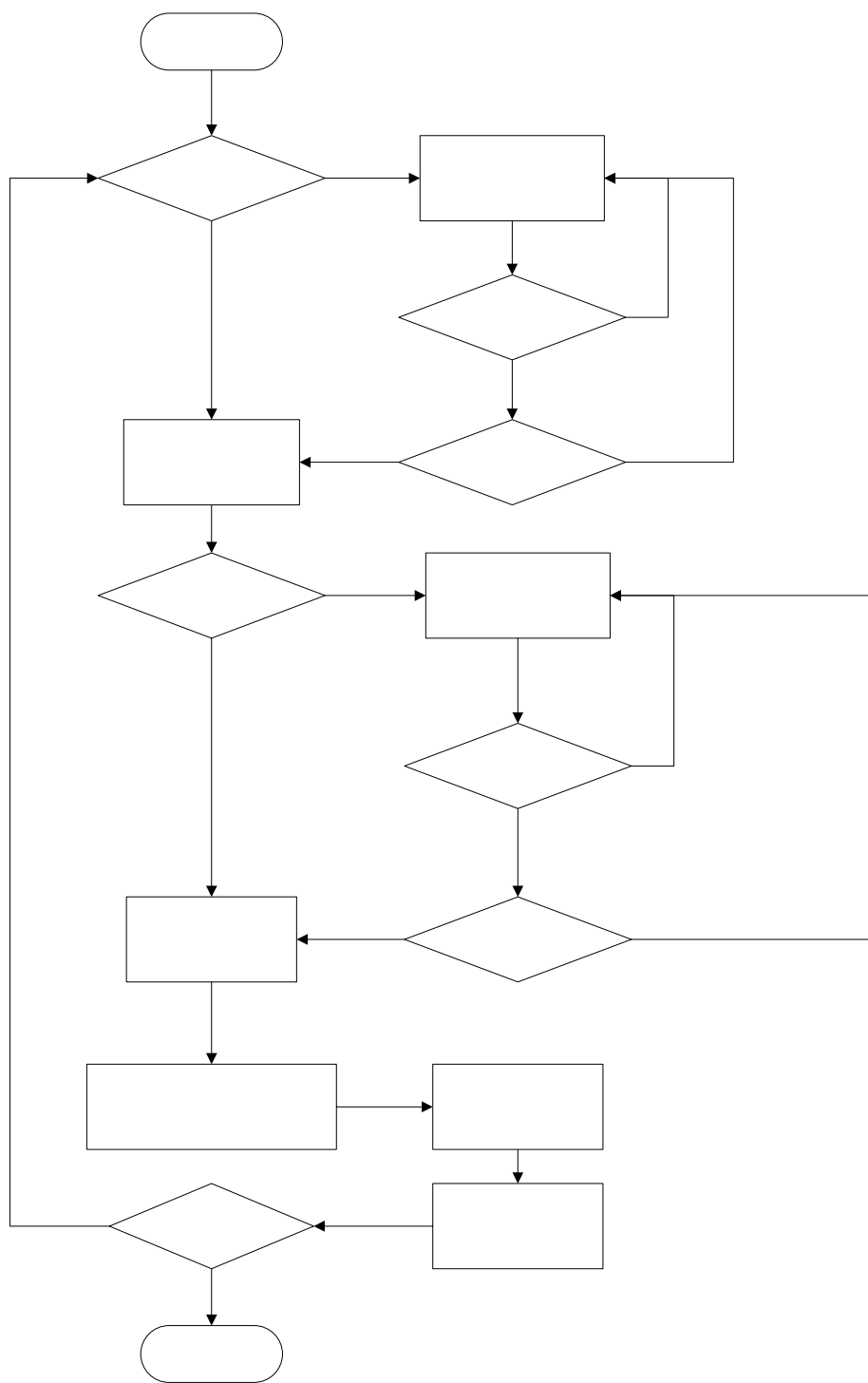
        infoList[k] = input.substring(j,i);
        k++;
        i++;
        j = i;
    }
}
}
}

```

- สำหรับผู้ใช้งานทั่วไปที่ไม่ใช่สมาชิกจะเรียกใช้งานเมธอด goNonmember() แทนการเรียกใช้ เมธอด goMember()
- ในเมธอด goNonmember() จะเรียกใช้เมธอดคอนสตรัคเตอร์ ในคลาส NonmemberScreen เพื่อแสดงผลหน้าเลือกประเภทของบริการ ได้แก่ บริการค้นหาเส้นทางจราจรและ บริการรายงานสภาพจราจร โดยพารามิเตอร์ที่ส่งเข้าไปคือออบเจกต์ ของคลาสหลักที่ใช้สำหรับแสดงค่าคอมโพเนนต์ต่างๆออกทางหน้าจอ ณ ปัจจุบัน
- เมื่อผู้ใช้งานเลือกบริการแล้ว จะใช้บริการต่างๆ เพื่อให้ได้ผลลัพธ์ของบริการ โดยสำหรับสมาชิกจะมีข้อมูลเส้นทาง ปลายทางและ เขตรายงาน ที่เป็นประโยชน์กับการใช้บริการ
- หลังจากที่ได้รับผลลัพธ์ของบริการแล้ว ผู้ใช้สามารถที่จะเข้าใช้บริการ ได้อีก หรือจะจบการทำงาน แอปพลิเคชันก็ได้

การทำงานของบริการค้นหาเส้นทางจราจร

ขั้นตอนการทำงานของบริการค้นหาเส้นทาง จะรวมการทำงานของผู้ใช้ที่เป็นสมาชิกกับผู้ใช้ทั่วไปเข้าด้วยกัน ซึ่งสมาชิกจะมีเส้นทางกับปลายทางให้อยู่แล้ว โดยการทำงานจะเป็นไปตามโฟลว์ชาร์ตดังรูปซึ่งมีขั้นตอนดังนี้



แสดงฟลว์ชาร์ตการทำงานของบริการค้นหาเส้นทางจราจร

เริ่มการพ

เลือกต

ไซ

- จุดเริ่มต้นของการใช้บริการ คือ การเลือกเส้นทาง ในที่นี้จะยกตัวอย่างผู้ใช้ที่เป็นสมาชิก เมื่อสมาชิกเลือกบริการค้นหาเส้นทางและกดปุ่ม Select ที่คลาส MemberScreen เมธอด `commandAction(Command c, Displayable d)` จะตรวจสอบเหตุการณ์ของการกดปุ่ม หาก `c` เท่ากับ `Select` จะตรวจสอบค่าดัชนีลำดับ หากค่าดัชนีลำดับเท่ากับ 0 หมายความว่า สมาชิกเลือกบริการค้นหาเส้นทาง ระบบจะมีการเก็บค่าข้อมูลต้นทางและปลายทางซึ่งเป็นข้อมูลสมาชิก เอาไว้ในตัวแปรอาร์เรย์ชื่อ `sourceList` และ `destList` ตามลำดับ จากนั้นจะเรียกใช้เมธอด `goSearchSource()` ของคลาสหลัก

```
public void commandAction(Command c, Displayable d) {

    int n;

    if(c == Select)    {

        n = getSelectedIndex();

        if(n == 0) {

            source.sourceList[0] = infoList[10]; //set source list for member

            source.sourceList[1] = infoList[12];

            dest.destList[0] = infoList[11]; //set destination list for member

            dest.destList[1] = infoList[13];

            midlet.goSearchsource();

        }

    }

}
```

- ที่เมธอด `goSearchSource()` จะประกาศออบเจกต์ของคลาส `SearchsourceScreen` เพื่อเรียกใช้ เมธอด `คอนสตรัคเตอร์ SearchsourceScreen(TrafficReport midlet)` ในการสร้างรายการค้นหา โดยนำค่าค้นหาในตัวแปรอาร์เรย์ `sourceList` แสดงผลออกทางหน้าจอ
- สมาชิกสามารถเลือกค้นหาจากรายการที่มีอยู่ หรือจะทำการค้นหารายการค้นหาใหม่ก็ได้ โดยการค้นหาจะรับค่าคีย์เวิร์ดค้นหาจากนั้นเรียกใช้เมธอด `goConnect(String input)` โดยค่าที่ส่งเข้าไปคือคีย์เวิร์ดที่ใช้ในการค้นหารายการค้นหา
- ข้อมูลจะถูกส่งไปยังไฟล์ `Server_RouteSearch.jsp` ที่เซิร์ฟเวอร์ เมื่อรับข้อมูลแล้วจะใช้คำสั่ง SQL ค้นหาโดยหารายการค้นหาที่ตรงกับคีย์เวิร์ด เมื่อค้นหาเจอจะต่อข้อมูลเป็นสตริงจากนั้นจึงส่งกลับไปยังแอปพลิเคชัน
- ที่โทรศัพท์เคลื่อนที่ เมื่อได้รับข้อมูลแล้วจะเรียกใช้เมธอด `คอนสตรัคเตอร์` ในคลาส `SearchsourceScreen` อีกครั้ง เพื่อสร้างลำดับรายการของค้นหาออกทางหน้าจอให้สมาชิกเลือก
- เมื่อสมาชิกเลือกค้นหาแล้วกดปุ่ม `Next` แล้วที่เมธอด `commandAction(Command c, Displayable d)` ตรวจสอบพบว่าเกิดเหตุการณ์กดปุ่ม `Next` ระบบจะเรียกใช้เมธอด `goConnect(String input)` ส่งข้อมูลค้นหา

ไปยังไฟล์ Server_RouteJunction.jsp เพื่อค้นหาแยกหรือโหนดที่อยู่ใกล้กับตำแหน่งของเส้นทางซึ่งอาจมีจำนวน 2 โหนด หรือ 1 โหนดก็ได้

- ที่เซิร์ฟเวอร์เมื่อได้รับข้อมูลต้นทางแล้ว ใช้คำสั่ง SQL ค้นหา และเมื่อได้ผลลัพธ์เป็นโหนดแล้วส่งกลับเป็นสตริงข้อมูล ไปยังแอปพลิเคชัน
- ที่โทรศัพท์เคลื่อนที่ เมื่อได้รับข้อมูลแล้ว เรียกใช้เมธอดชื่อ getrouteArray(result) โดยค่า result คือ ค่าโหนดต้นทาง เพื่อนำค่าโหนดไปเก็บยังตัวแปรอาร์เรย์ routeArray

```
public void getrouteArray(String input) {  
    for(int i=0;i<input.length();i++) {  
        if(input.charAt(i) == ',') {  
            if(input.charAt(i+1) == '$') {  
                routeArray[k] = input.substring(j,i);  
                i = 300;  
            }  
            else {  
                routeArray[k] = input.substring(j,i);  
                k++;  
                i++;  
                j = i;  
            }  
        }  
    }  
}
```

- จากนั้นระบบจะเรียกใช้เมธอดของคลาสหลักชื่อ goSearchdest()
- ภายในเมธอด goSearchdest() จะประกาศออบเจกต์ของคลาส SearchdestScreen เพื่อเรียกใช้ เมธอดคอนสตรักเตอร์ SearchdestScreen(TrafficReport midlet) ในการสร้างรายการปลายทาง โดยนำค่าปลายทางในตัวแปรอาร์เรย์ destList แสดงผลออกทางหน้าจอ
- หลังจากนั้นการค้นหาหรือการเลือกปลายทาง รวมทั้งการส่งข้อมูลไปยังไฟล์ที่เซิร์ฟเวอร์ ขั้นตอนจะเป็นเหมือนกับการค้นหาและการเลือกต้นทาง
- เมื่อสมาชิกกดปุ่ม Next แล้ว ขั้นตอนต่อไประบบจะเรียกใช้เมธอด goCost()

- ภายในเมธอด goCost() จะประกาศออบเจ็กต์ของคลาส CostScreen เพื่อเรียกใช้เมธอดคอนสตรักเตอร์ CostScreen(TrafficReport midlet) ในการสร้างรายการปัจจัยในการค้นหาเส้นทาง โดยสมาชิกสามารถเลือกได้หลายปัจจัย ได้แก่ เวลา เงินและระยะทาง แสดงผลออกทางหน้าจอ
- เมื่อสมาชิกเลือกปัจจัยได้แล้วกดปุ่ม Route ที่เมธอด commandAction(Command c, Displayable d) จะตรวจพบเหตุการณ์กดปุ่ม Route แล้วโปรแกรมจะเรียกใช้เมธอด goConnect(String input) โดยส่งข้อมูลค้นหาเส้นทางปลายทางและปัจจัย ไปยังไฟล์ Server_RouteResponse.jsp

```

if(c == Route) {

    try{

        for(int i=0;i<=size()-1;i++) {

            if(isSelected(i))

                source.routeArray[i+4] = "y";

            else

                source.routeArray[i+4] = "n";

        }

        if((source.routeArray[0].equals(source.routeArray[1]))||(source.routeArray[2].equals(source.routeArray[3]
)))

            midlet.goError("cost");

        else {

            encode =

midlet.EncodeConnection("source1="+source.routeArray[0]+"&source2="+source.routeArray[1]+"&dest1="+
source.routeArray[2]+

            "&dest2="+source.routeArray[3]+"&time="+source.routeArray[4]+"&money="+source.routeArray[5]+"
&distance="+source.routeArray[6]);

            serverURL =

"http://localhost:8080/Server_RouteResponse.jsp?" + encode;

            System.out.println(serverURL);

            result = midlet.goConnect(serverURL);

            result = result.trim();

            System.out.println(result);

            midlet.goResponse(result,1);

        }

    }

```

```

        }
        catch(IOException e) {
        }
    }
}

```

- ที่ไฟล์ Server_RouteResponse.jsp เมื่อได้รับข้อมูลแล้ว เรียกใช้빈เพื่อติดต่อกับคลาสประมวลผลค้นหาเส้นทาง โดยส่งต้นทางและปลายทางไปที่ละคู่เมื่อได้ผลลัพธ์จนครบ เปรียบเทียบหาเส้นทางที่ดีที่สุด จากปัจจัยที่ได้รับมา แล้วส่งเส้นทางนั้นกลับไปยังแอปพลิเคชัน

```

<jsp:useBean id="Route1" class="TrafficReport.routing" />
<%
    String reqsource1 = request.getParameter("source1");
    String reqsource2 = request.getParameter("source2");
    String reqdest1 = request.getParameter("dest1");
    String reqdest2 = request.getParameter("dest2");
    String reqtime = request.getParameter("time");
    String reqmoney = request.getParameter("money");
    String reqdistance = request.getParameter("distance");

    String res = new String();
    String res1 = new String();
    String res2 = new String();
    String res3 = new String();
    String res4 = new String();

    //array[0]=time;array[1]=money;array[2]=distance

    double fact1[] = new double[3];
    double fact2[] = new double[3];
    double fact3[] = new double[3];
    double fact4[] = new double[3];
    double comparetime[] = new double[4];
    double comparemoney[] = new double[4];
    double comparedistance[] = new double[4];

    double temptime = 99999.99;
    double tempmoney = 99999.99;

```

```

double tempdistance = 9999999.99;

int count = 0;

int path_time = 0;

int path_money = 0;

int path_distance = 0;

//-----First Sending-----

Route1.setRoute(reqsource1,reqdest1,reqmoney,reqtime,reqdistance);

res1 = Route1.getRoute();

res1 = res1.trim();

fact1 = Route1.getFactor();

if(res1.equals("Error")) {

    fact1[0] = 99999.99;

    fact1[1] = 99999.99;

    fact1[2] = 9999999.99;

}

count = 1;

comparetime[0] = fact1[0];

comparemoney[0] = fact1[1];

comparedistance[0] = fact1[2];

System.out.println(res1);

System.out.println(fact1[0]);

System.out.println(fact1[1]);

System.out.println(fact1[2]);

if((!reqsource2.equals("null"))||(!reqdest2.equals("null"))) {

//-----Second Sending-----

    if(reqsource2.equals("null"))

        Route1.setRoute(reqsource1,reqdest2,reqmoney,reqtime,reqdistance);

    else if(reqdest2.equals("null"))

        Route1.setRoute(reqsource2,reqdest1,reqmoney,reqtime,reqdistance);

    res2 = Route1.getRoute();

    res2 = res2.trim();

    fact2 = Route1.getFactor();

    if(res2.equals("Error")) {

        fact2[0] = 99999.99;

```

```

        fact2[1] = 99999.99;
        fact2[2] = 9999999.99;
    }
    count = 2;
    comparetime[1] = fact2[0];
    comparemoney[1] = fact2[1];
    comparedistance[1] = fact2[2];
    System.out.println(res2);
    System.out.println(fact2[0]);
    System.out.println(fact2[1]);
    System.out.println(fact2[2]);

//-----Third Sending-----
    if(!reqsource2.equals("null"))&&(!reqdest2.equals("null")) {
        Route1.setRoute(reqsource2,reqdest1,reqmoney,reqtime,reqdistance);
        res3 = Route1.getRoute();
        res3 = res3.trim();
        fact3 = Route1.getFactor();
        if(fact3.equals("Error")){
            fact3[0] = 99999.99;
            fact3[1] = 99999.99;
            fact3[2] = 9999999.99;
        }
        count = 3;
        comparetime[2] = fact3[0];
        comparemoney[2] = fact3[1];
        comparedistance[2] = fact3[2];
        System.out.println(res3);
        System.out.println(fact3[0]);
        System.out.println(fact3[1]);
        System.out.println(fact3[2]);

//-----Fourth Sending-----
        Route1.setRoute(reqsource2,reqdest2,reqmoney,reqtime,reqdistance);
        res4 = Route1.getRoute();
        res4 = res4.trim();

```

```

        fact4 = Route1.getFactor();
        if(fact4.equals("Error")){
            fact4[0] = 99999.99;
            fact4[1] = 99999.99;
            fact4[2] = 9999999.99;
        }
        count = 4;
        comparetime[3] = fact4[0];
        comparemoney[3] = fact4[1];
        comparedistance[3] = fact4[2];
        System.out.println(res4);
        System.out.println(fact4[0]);
        System.out.println(fact4[1]);
        System.out.println(fact4[2]);
    }
}

//-----Consideration-----
//consider with factor's priority so we give the priority of money the first,priority of time is second
//and priority of distance the last
for(int i=0;i<count;i++){
    if(comparetime[i] <= temptime){
        temptime = comparetime[i];
        path_time = i;
    }
}
System.out.println(temptime);
for(int j=0;j<count;j++){
    if(comparemoney[j] <= tempmoney){
        tempmoney = comparemoney[j];
        path_money = j;
    }
}
System.out.println(tempmoney);
for(int k=0;k<count;k++) {

```

```

        if(comparedistance[k] <= tempdistance) {
            tempdistance = comparedistance[k];
            path_distance = k;
        }
    }

    System.out.println(tempdistance);
    System.out.println(reqtime);
    System.out.println(reqmoney);
    System.out.println(reqdistance);

    if(((reqtime.equals("y"))&&(reqmoney.equals("n"))&&(reqdistance.equals("n")))||
        ((reqtime.equals("y"))&&(reqmoney.equals("n"))&&(reqdistance.equals("y"))))) {
        System.out.println(path_time);
        if(path_time == 0)
            out.println(res1+"time="+fact1[0]+",money="+fact1[1]+",distance="+fact1[2]);
        else if(path_time == 1)
            out.println(res2+"time="+fact2[0]+",money="+fact2[1]+",distance="+fact2[2]);
        else if(path_time == 2)
            out.println(res3+"time="+fact3[0]+",money="+fact3[1]+",distance="+fact3[2]);
        else if(path_time == 3)
            out.println(res4+"time="+fact4[0]+",money="+fact4[1]+",distance="+fact4[2]);
    }

    else if(((reqtime.equals("n"))&&(reqmoney.equals("y"))&&(reqdistance.equals("n")))||
        ((reqtime.equals("n"))&&(reqmoney.equals("y"))&&(reqdistance.equals("y")))||
        ((reqtime.equals("y"))&&(reqmoney.equals("y"))&&(reqdistance.equals("n")))||
        ((reqtime.equals("y"))&&(reqmoney.equals("y"))&&(reqdistance.equals("y")))||
        ((reqtime.equals("n"))&&(reqmoney.equals("n"))&&(reqdistance.equals("n"))))) {
        System.out.println(path_money);
        if(path_money == 0)
            out.println(res1+"time="+fact1[0]+",money="+fact1[1]+",distance="+fact1[2]);
        else if(path_money == 1)
            out.println(res2+"time="+fact2[0]+",money="+fact2[1]+",distance="+fact2[2]);
        else if(path_money == 2)
            out.println(res3+"time="+fact3[0]+",money="+fact3[1]+",distance="+fact3[2]);
        else if(path_money == 3)

```



```

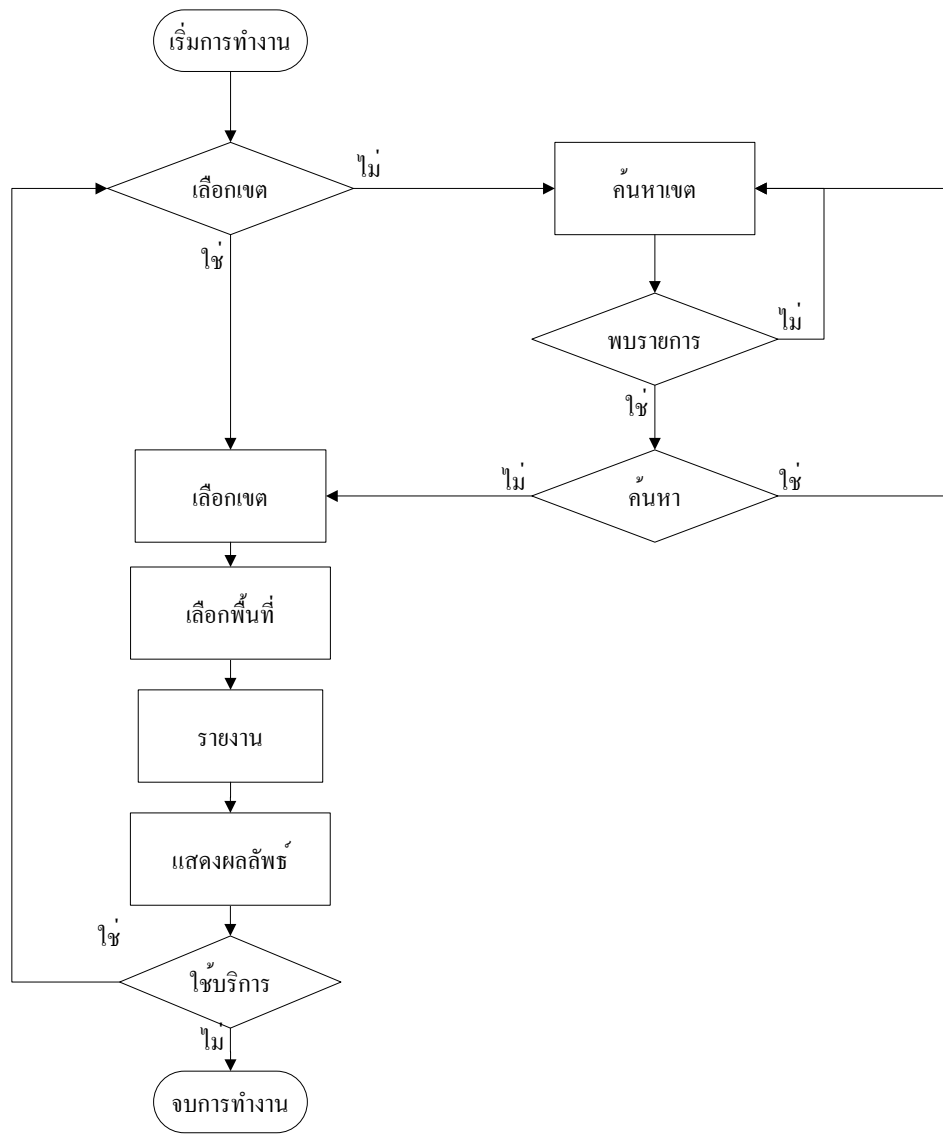
        out.println(res4+",time="+fact4[0]+",money="+fact4[1]+",distance="+fact4[2]);
    }
    else if((reqtime.equals("n"))&&(reqmoney.equals("n"))&&(reqdistance.equals("y"))){
        System.out.println(path_distance);
        if(path_distance == 0)
            out.println(res1+",time="+fact1[0]+",money="+fact1[1]+",distance="+fact1[2]);
        else if(path_distance == 1)
            out.println(res2+",time="+fact2[0]+",money="+fact2[1]+",distance="+fact2[2]);
        else if(path_distance == 2)
            out.println(res3+",time="+fact3[0]+",money="+fact3[1]+",distance="+fact3[2]);
        else if(path_distance == 3)
            out.println(res4+",time="+fact3[0]+",money="+fact3[1]+",distance="+fact4[2]);
    }
}
%>

```

- ที่แอปพลิเคชันเมื่อได้รับข้อมูลแล้ว เรียกใช้เมธอดของคลาสหลักชื่อ goResponse(String input,int flag)
- ภายในเมธอด goResponse(String input, int flag) จะประกาศออบเจกต์ของคลาส ResponseScreen เพื่อเรียกใช้เมธอดคอนสตรักเตอร์ ResponseScreen(TrafficReport midlet, String input,int flag) ในการนำข้อมูลผลลัพธ์เส้นทาง แสดงผลออกทางหน้าจอ
- สำหรับค่าพารามิเตอร์ flag จะเก็บไว้ในตัวแปร checkResponse สำหรับตรวจสอบว่าเมื่อผู้ใช้กดปุ่ม Back แล้ว ถ้าค่า checkResponse เท่ากับ 1 หมายความว่าให้กลับไปหน้าจอเลือกปัจจัยค้นหาเส้นทาง ในบริการค้นหาเส้นทาง หากค่า checkResponse เท่ากับ 2 หมายความว่าให้กลับไปหน้าจอเลือกพื้นที่รายงาน ในบริการรายงานสภาพจราจร

การทำงานของบริการรายงานสภาพการจราจร

ขั้นตอนการทำงานของบริการรายงานสภาพการจราจร จะรวมการทำงานของผู้ใช้ที่เป็นสมาชิกกับผู้ใช้ทั่วไปเข้าด้วยกัน ซึ่งสมาชิกจะมีเขตรายงานให้อยู่แล้ว โดยการทำงานจะเป็นไปตามโฟลว์ชาร์ตดังรูปซึ่งมีขั้นตอนดังนี้



แสดงโฟลว์ชาร์ตการทำงานของบริการรายงานสภาพจราจร

- จุดเริ่มต้นของการใช้บริการ คือ การเลือกเขตรายงาน ในที่นี้จะยกตัวอย่างผู้ให้บริการทั่วไป เมื่อผู้ใช้เลือกบริการรายงานสภาพจราจรและกดปุ่ม Select ที่คลาส NonmemberScreen เมธอด `commandAction(Command c, Displayable d)` จะตรวจสอบเหตุการณ์ของการกดปุ่ม หาก c เท่ากับ Select จะตรวจสอบค่าดัชนีลำดับ หากค่าดัชนีลำดับเท่ากับ 1 หมายความว่า ผู้ใช้เลือกบริการรายงานสภาพจราจร ระบบจะเรียกใช้เมธอด `goReport()` ของคลาสหลัก
- ภายในเมธอด `goReport()` จะประกาศออบเจกต์ของคลาส `ReportareaScreen` เพื่อเรียกใช้เมธอดคอนสตรัคเตอร์ `ReportareaScreen(TrafficReport midlet)` ในการสร้างรายการเขตรายงาน แสดงผลออกทางหน้าจอ

- สำหรับผู้ใช้บริการทั่วไป จะไม่มีข้อมูลเขตราชงานมาให้เหมือนกับสมาชิก จึงเริ่มด้วยการค้นหาโดยรับค่าคีย์เวิร์ดเขตราชงาน จากนั้นเรียกใช้เมธอด goConnect(String input) โดยค่าที่ส่งเข้าไปคือคีย์เวิร์ดที่ใช้ในการค้นหารายการเขตราชงาน
- ข้อมูลจะถูกส่งไปยังไฟล์ Server_AreaSearch.jsp ที่เซิร์ฟเวอร์ เมื่อรับข้อมูลแล้วจะใช้คำสั่ง SQL ค้นหา โดยหารายการเขตราชงานที่ตรงกับคีย์เวิร์ด เมื่อค้นหาเจอจะต่อข้อมูลเป็นสตรีมจากนั้นจึงส่งกลับไปยังแอปพลิเคชัน
- ที่โทรศัพท์เคลื่อนที่ เมื่อได้รับข้อมูลแล้วจะเรียกใช้เมธอดคอนสตรัคเตอร์ในคลาส ReportareaScreen อีกครั้ง เพื่อสร้างลำดับรายการเขตราชงานออกทางหน้าจอให้ผู้ใช้เลือก
- เมื่อผู้ใช้เลือกเขตราชงานแล้วกดปุ่ม Next ที่เมธอด commandAction(Command c, Displayable d) ตรวจสอบพบว่าการกดปุ่ม Next ระบบจะเรียกใช้เมธอด goConnect(String input) ส่งข้อมูลเขตราชงานไปยังไฟล์ Server_RangeSearch.jsp เพื่อค้นหาพื้นที่ที่มีการราชงานในเขตราชงานนั้น
- ที่เซิร์ฟเวอร์เมื่อได้รับข้อมูลเขตราชงานแล้ว ใช้คำสั่ง SQL ค้นหา และเมื่อได้ผลลัพธ์เป็นพื้นที่ราชงานจะส่งกลับเป็นสตรีมข้อมูล ไปยังแอปพลิเคชัน
- ที่โทรศัพท์เคลื่อนที่ เมื่อได้รับข้อมูลแล้ว เรียกใช้เมธอดชื่อ getRangelist(result) โดยค่า result คือ ค่าพื้นที่ราชงาน เพื่อนำค่าไปเก็บยังตัวแปรอาร์เรย์ rangeList

```

public void getRangelist(String input) {
    for(int i=0;i<input.length();i++) {
        if(input.charAt(i) == ',') {
            if(input.charAt(i+1) == '$') {
                range.rangeList[k] = input.substring(j,i);
                i = input.length();
            }
            else {
                range.rangeList[k] = input.substring(j,i);
                k++;
                i++;
                j = i;
            }
        }
        for(int l=k+1;l<50;l++)
            range.rangeList[l] = null;
    }
}

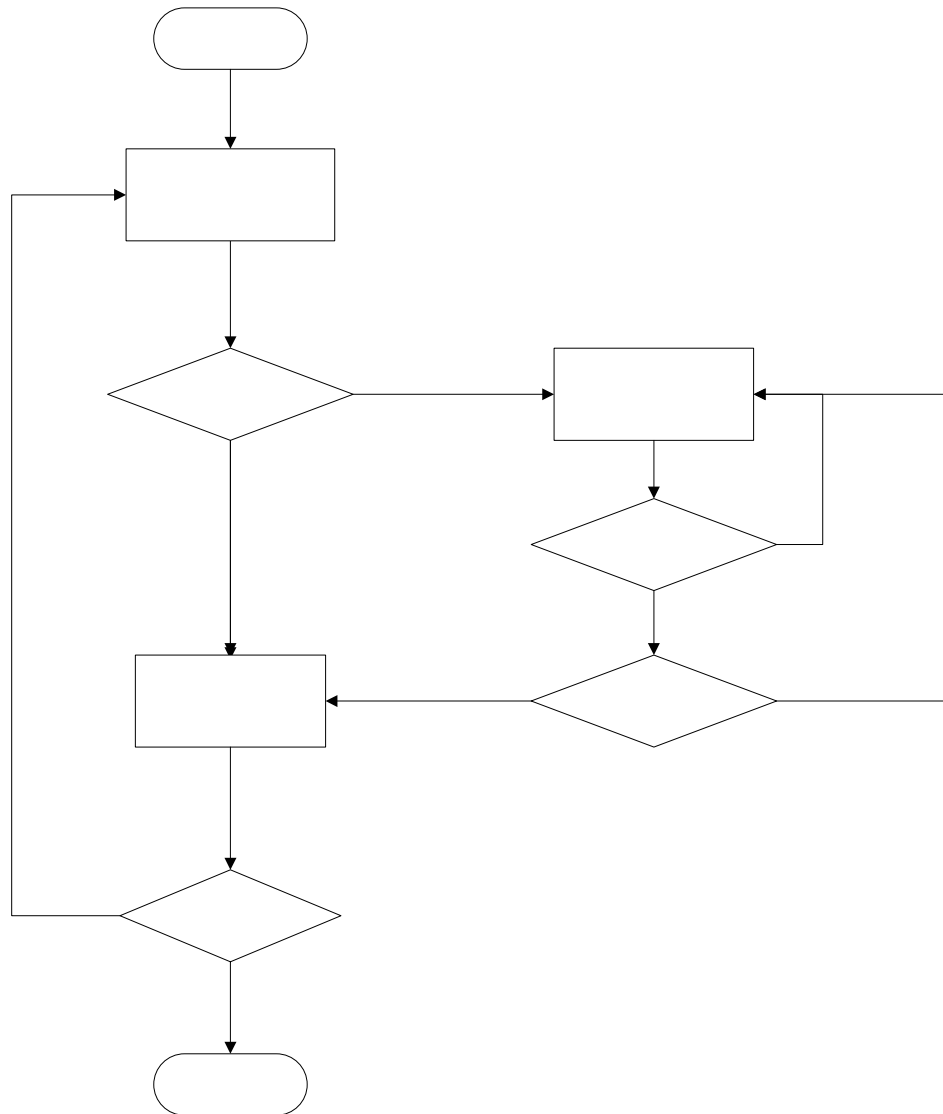
```

```
}
```

- จากนั้นระบบจะเรียกใช้เมธอดของคลาสหลักชื่อ goRange()
- ภายในเมธอด goRange() จะประกาศออบเจกต์ของคลาส ReportrangeScreen เพื่อเรียกใช้ เมธอดคอนสตรักเตอร์ ReportrangeScreen(TrafficReport midlet) ในการสร้างรายการพื้นที่รายงาน โดยนำค่าพื้นที่รายงานในตัวแปรอาร์เรย์ rangeList แสดงผลออกทางหน้าจอ
- เมื่อผู้ใช้เลือกพื้นที่รายงานแล้วกดปุ่ม Report ที่เมธอด commandAction(Command c, Displayable d) ตรวจสอบพบว่าการกดปุ่ม Report ระบบจะเรียกใช้เมธอด goConnect(String input) ส่งข้อมูลพื้นที่รายงานไปยังไฟล์ Server_ReportResponse.jsp เพื่อค้นหารายงานสภาพจราจรในพื้นที่รายงานนั้น
- ที่เซิร์ฟเวอร์เมื่อได้รับข้อมูลเขตรายงานแล้ว ใช้คำสั่ง SQL ค้นหา และเมื่อได้ผลลัพธ์เป็นรายละเอียดของรายงานสภาพจราจร จะส่งกลับเป็นสตรีมข้อมูล ไปยังแอปพลิเคชัน
- ที่แอปพลิเคชันเมื่อได้รับข้อมูลแล้ว เรียกใช้เมธอดของคลาสหลักชื่อ goResponse(String input, int flag)
- ภายในเมธอด goResponse(String input, int flag) จะประกาศออบเจกต์ของคลาส ResponseScreen เพื่อเรียกใช้เมธอดคอนสตรักเตอร์ ResponseScreen(TrafficReport midlet, String input, int flag) ในการนำข้อมูลผลลัพธ์สภาพจราจร แสดงผลออกทางหน้าจอ
- สำหรับค่าพารามิเตอร์ flag จะเก็บไว้ในตัวแปร checkResponse สำหรับตรวจสอบว่าเมื่อผู้ใช้กดปุ่ม Back แล้ว ถ้าค่า checkResponse เท่ากับ 1 หมายความว่าให้กลับไปหน้าจอเลือกปัจจัยค้นหาเส้นทาง ในบริการค้นหาเส้นทาง หากค่า checkResponse เท่ากับ 2 หมายความว่าให้กลับไปหน้าจอเลือกพื้นที่รายงาน ในบริการรายงานสภาพจราจร

การทำงานของบริการแก้ไขข้อมูลสมาชิก

ขั้นตอนการทำงานของบริการแก้ไขข้อมูล จะเป็นบริการที่เพิ่มเข้ามาสำหรับสมาชิกเท่านั้น โดยการทำงานจะเป็นไปตามโฟลว์ชาร์ตดังรูปซึ่งมีขั้นตอนดังนี้



เริ่มการ

แก้ไขข้อ

แสดงโฟลว์ชาร์ตการทำงานของบริการแก้ไขข้อมูลสมาชิก

- จุดเริ่มต้นของการใช้บริการ คือ การแก้ไขข้อมูลทั่วไปในแต่ละฟิลด์ ซึ่งประกอบด้วย ข้อมูลชื่อสมาชิก, รหัสผ่าน, ชื่อสมาชิกจริง, อีเมล, ที่อยู่, ชื่อผู้ให้บริการ, เบอร์โทรศัพท์เคลื่อนที่, ชื่อผู้ใช้งานโทรศัพท์, รหัสผ่านโทรศัพท์
- เมื่อสมาชิกเลือกบริการแก้ไขข้อมูลและกดปุ่ม Select ที่คลาส MemberScreen เมธอด `commandAction(Command c, Displayable d)` จะตรวจสอบเหตุการณ์ของการกดปุ่ม หาก c เท่ากับ Select จะตรวจสอบค่าดัชนีลำดับ หากค่าดัชนีลำดับเท่ากับ 2 หมายความว่า สมาชิกเลือกบริการแก้ไขข้อมูล ระบบจะเรียกใช้เมธอด `goEdituser()` ของคลาสหลัก

เลือก

ใช้

- ภายในเมธอด goEdituser() จะประกาศออบเจกต์ของคลาส EdituserScreen เพื่อเรียกใช้เมธอดคอนสตรัคเตอร์ EdituserScreen(TrafficReport midlet) ในการสร้างรายการข้อมูลต่างๆ ของสมาชิก แสดงผลออกทางหน้าจอ
- เมื่อผู้ใช้บริการแก้ไขข้อมูลต่างๆ เสร็จแล้วกดปุ่ม Next ที่คลาส EdituserScreen เมธอด commandAction(Command c, Displayable d) จะตรวจสอบเหตุการณ์ของการกดปุ่ม หาก c เท่ากับ Next จะมีการเปลี่ยนแปลงค่าข้อมูลในอาร์เรย์ infoList เป็นข้อมูลเตรียมแก้ไข จากนั้นระบบจะเรียกใช้เมธอด goEditdistrict() ของคลาสหลัก

```

if(c == Next) {

    if((pmet[0] == null)&&(pmet[1] == null))
        midlet.goError("edit");
    else if(!(pmet[0].equals(pmet[1])))
        midlet.goError("edit");
    else {

        n = provChoice.getSelectedIndex();
        mem.infoList[1] = pmet[1];
        mem.infoList[2] = pmet[2];
        mem.infoList[3] = pmet[3];
        mem.infoList[5] = pmet[4];
        mem.infoList[6] = pmet[5];
        mem.infoList[7] = provChoice.getString(n);
        mem.infoList[8] = pmet[6];
        mem.infoList[9] = pmet[7];
        distr.distrList[0] = mem.infoList[4];
        midlet.goEditdistrict();

    }

}

```

- ภายในเมธอด goEditdistrict() จะประกาศออบเจกต์ของคลาส EditdistrictScreen เพื่อเรียกใช้ เมธอดคอนสตรัคเตอร์ EditdistrictScreen(TrafficReport midlet) ในการสร้างรายการเขตรายงาน แสดงผลออกทางหน้าจอ
- ในขั้นตอนนี้จะเหมือนกับการค้นหารายการเขตรายงาน

- เมื่อสมาชิกเลือกเขตรายงานแล้วกดปุ่ม Edit จะมีการเปลี่ยนแปลงค่าข้อมูลในอาร์เรย์ infoList เป็นข้อมูลเขตรายงานเตรียมแก้ไข จากนั้นระบบจะเรียกใช้เมธอด goConnect(String input) ของคลาสหลักโดยค่า input คือค่าข้อมูลที่สมาชิกจะแก้ไขทั้งหมดส่งไปยังไฟล์ Server_EditUpdate.jsp

```

if(c == Edit) {

    try{

        n = distrChoice.getSelectedIndex();

        mem.infoList[4] = distrChoice.getString(n);

        encode =

        midlet.EncodeConnection("user="+mem.infoList[0]+"&passw="+mem.infoList[1]+"&nam="+mem.infoList[2]
        ]+"&ema="+mem.infoList[3]

        +"&dist="+mem.infoList[4]+"&add="+mem.infoList[5]+"&mobile="+mem.infoList[6]+"&pro="+mem.i
        nfoList[7]+"&mousr="+mem.infoList[8]+"&mopass="+mem.infoList[9]);

        serverURL = "http://localhost:8080/Server_EditUpdate.jsp?"+encode;

        result = midlet.goConnect(serverURL);

        result = result.trim();

        if(result.equals("ok")) {

            log.usr_pss =

            midlet.EncodeConnection("username="+mem.infoList[0]+"&password="+mem.infoList[1]);

            midlet.goEditok();

        }

        else

            midlet.goError("edit");

    }

    catch(IOException e) {

    }

}

```

- ที่เซิร์ฟเวอร์ เมื่อรับข้อมูลแล้วจะใช้คำสั่ง SQL ในการอัปเดตข้อมูลสมาชิก โดยที่ชื่อ และรหัสผ่านสมาชิกจะต้องตรงกันกับในฐานข้อมูล
- ที่โทรศัพท์เคลื่อนที่ เมื่อได้รับข้อมูลแล้วจะเรียกใช้เมธอด Editok() ของคลาสหลัก เพื่อสร้างผลลัพธ์ให้ผู้ใช้ทราบว่าการอัปเดตข้อมูลสมาชิกใหม่เสร็จสิ้นแล้ว